

Reduced Order Models

Riccardo Rossi

Nicolás Sibuet

Sebastian Ares de Parga

Raul Bravo

Marco Zuñiga

Carlos Roig

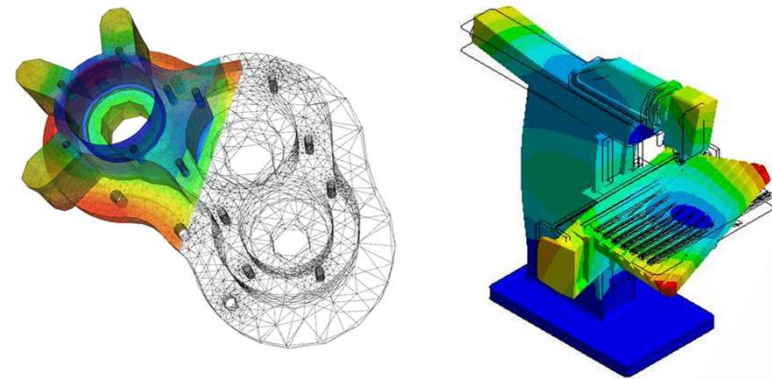
Ruben Zorrilla



FEM:

Parametrized PDE systems on
discretized geometry and time

Entirely physics-based
State-of-the-art accuracy for
complex problems



BUT

Computationally expensive:
requires abundant time and
resources

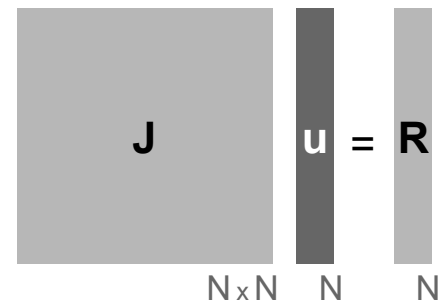


Where to cut costs?

During solving process: Newton-Raphson problem (or alike) of the form

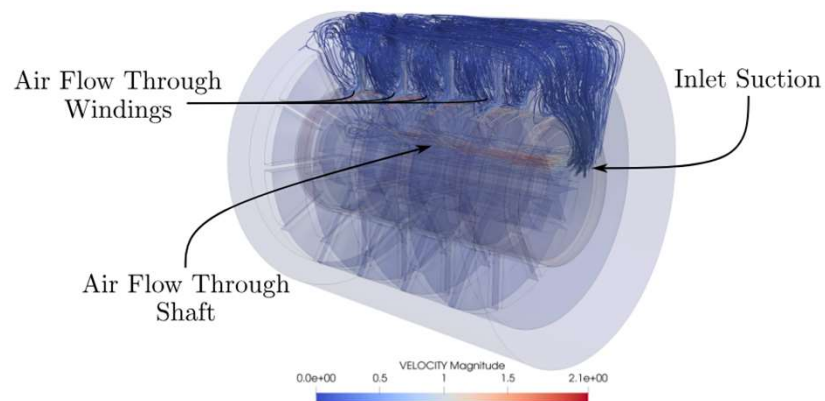
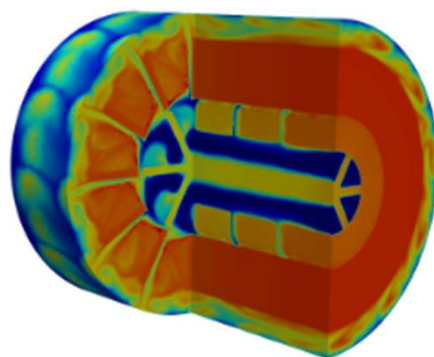
$$J_t^k \delta \mathbf{u}_t^k = \mathbf{R}_t^k$$




$$\mathbf{J} \mathbf{u} = \mathbf{R}$$

$N \times N$ N N





$$\mathbf{J} \quad \mathbf{u} = \mathbf{R}$$

$N \times N \quad N \quad N$

$N = 843817$

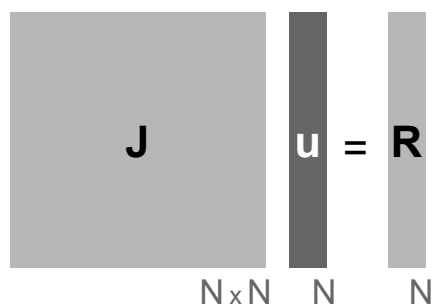
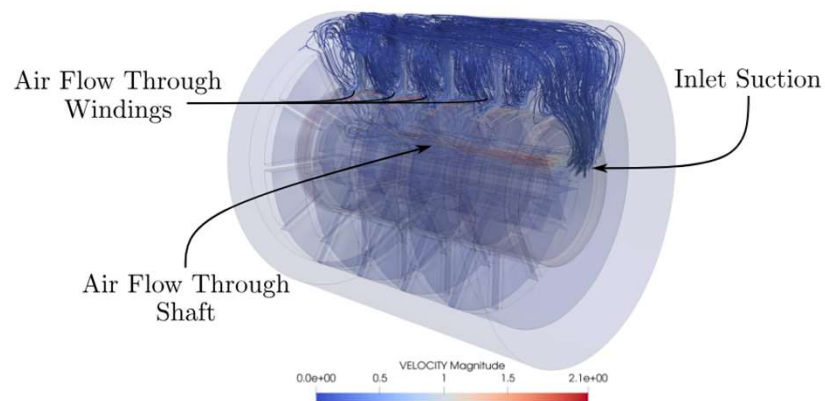
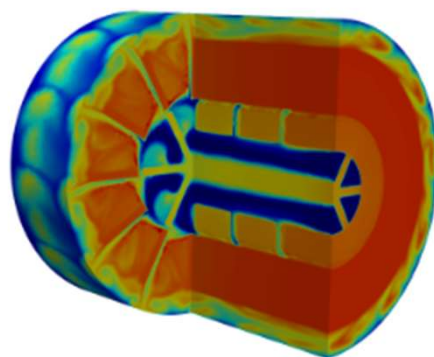
}

21 hours

Computed on 20 CPUs

[1] Ares de Parga, S., Bravo, J. R., et. al. (2024). Parallel Reduced Order Modeling for Digital Twins using High-Performance Computing Workflows. *arXiv preprint arXiv:2409.09080*.





$N = 843817$

}

21 hours

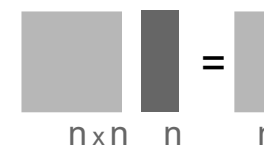
ROM



$n = 44$

}

27 minutes



Computed on 20 CPUs

[1] Ares de Parga, S., Bravo, J. R., et. al. (2024). Parallel Reduced Order Modeling for Digital Twins using High-Performance Computing Workflows. *arXiv preprint arXiv:2409.09080*.

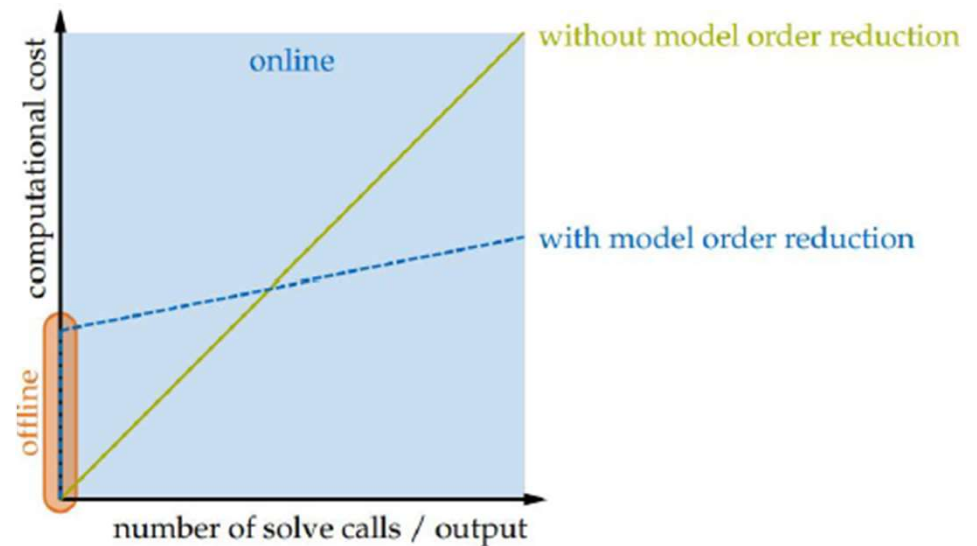


How is this achieved?

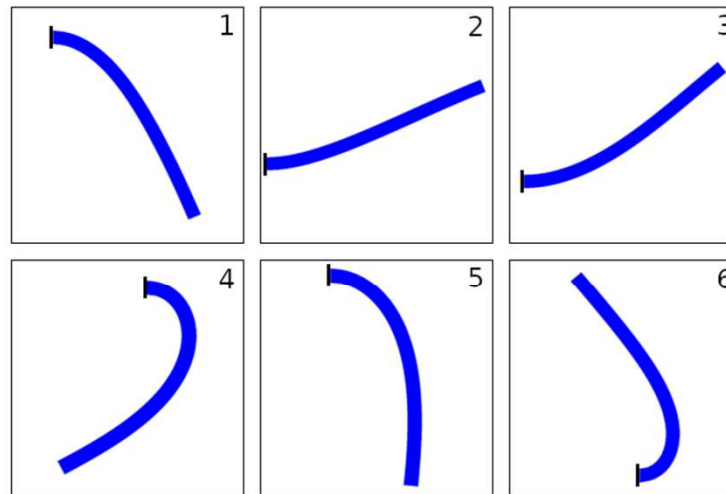
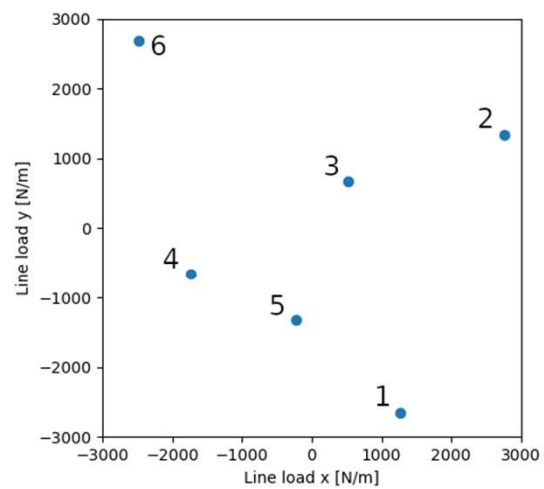
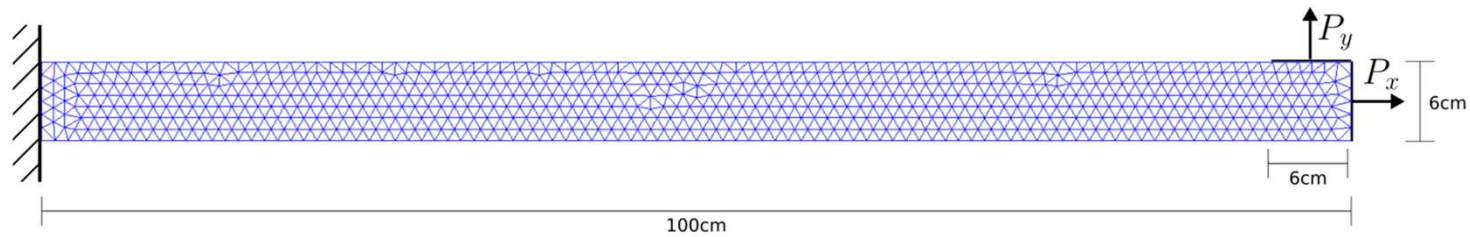
Data-based methods

OFFLINE: Gather and process data from a few FOM models

ONLINE: Use this data to help run new simulations faster

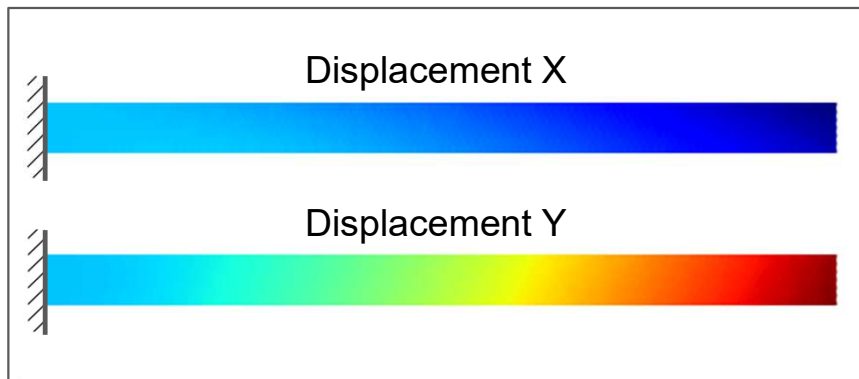


Structural problem: static, hyperelastic



OFFLINE:

Given $\mu_i = (P_{x,i}, P_{y,i})$. Solve FOM:



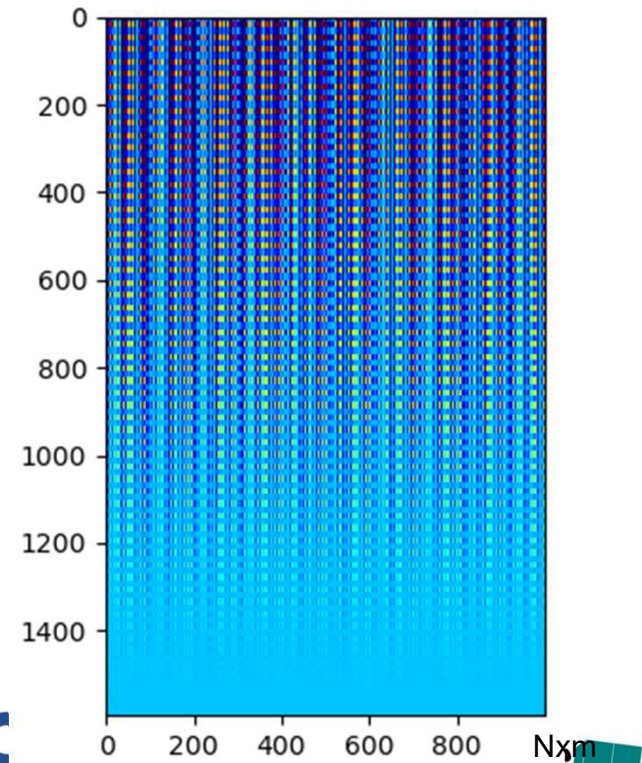
Flatten

Snapshot (u_i)

$N \times 1$

Gather for m
values of μ

Snapshots matrix (S)



MULTI-PHYSICS

OFFLINE:

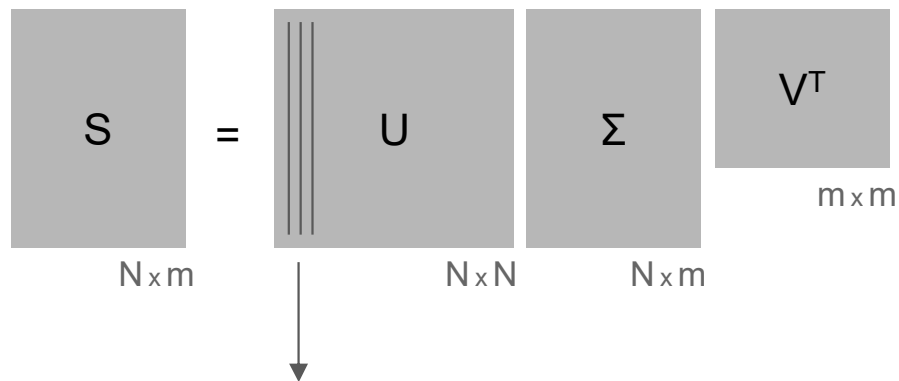
Decompose snapshots matrix via SVD

$$\begin{array}{c} \boxed{S} \\ N \times m \end{array} = \begin{array}{c} \boxed{U} \\ N \times N \end{array} \begin{array}{c} \boxed{\Sigma} \\ N \times m \end{array} \begin{array}{c} \boxed{V^T} \\ m \times m \end{array}$$



OFFLINE:

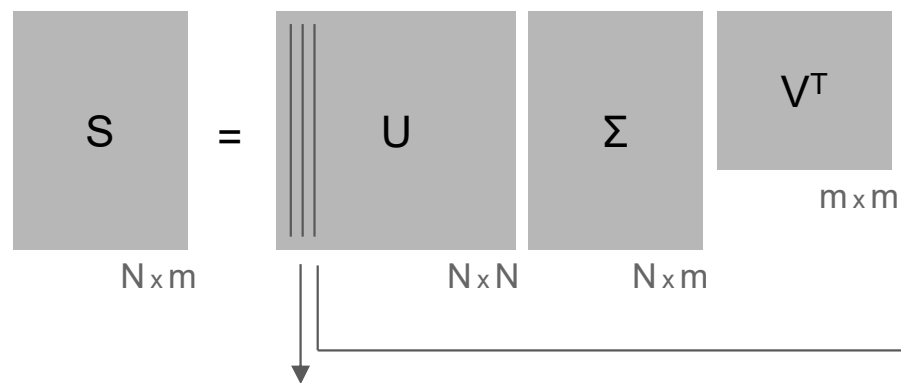
Decompose snapshots matrix via SVD



- Each column in U is a mode for the full snapshots in S .
- All modes are orthonormal:
$$U^T U = I$$
- Modes are ordered in terms of importance.

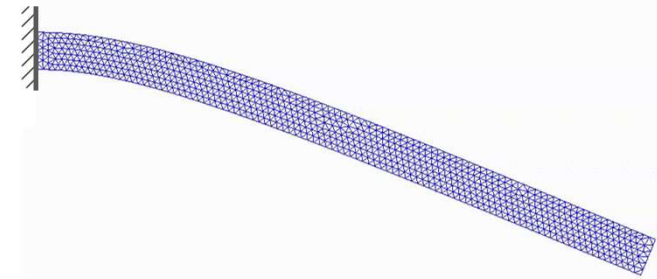
OFFLINE:

Decompose snapshots matrix via SVD

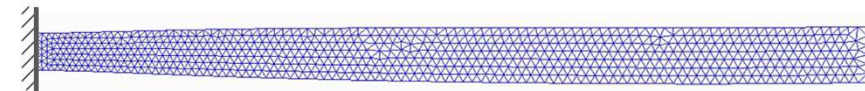


- Each column in U is a mode for the full snapshots in S .
- All modes are orthonormal:
$$U^T U = I$$
- Modes are ordered in terms of importance.

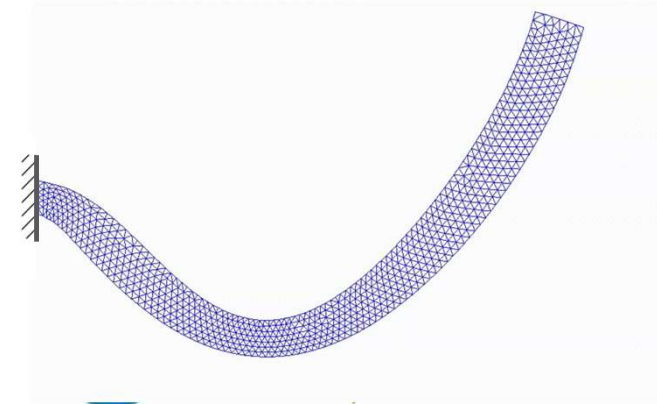
Mode 1



Mode 2



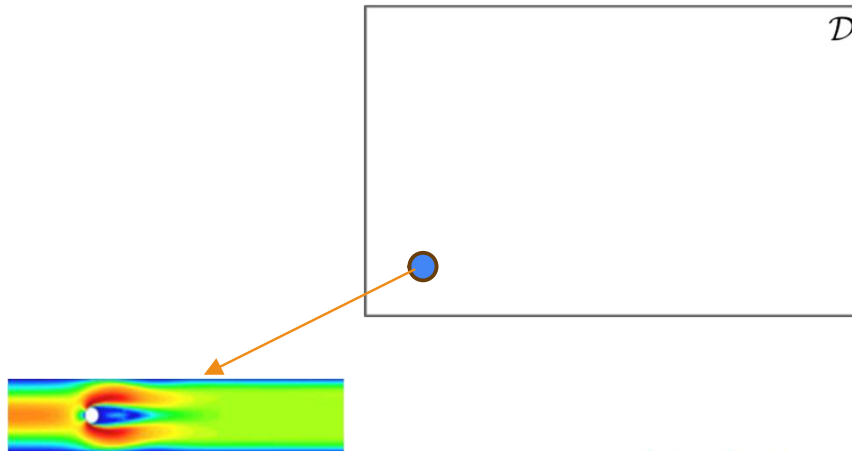
Mode 3



Time dependent

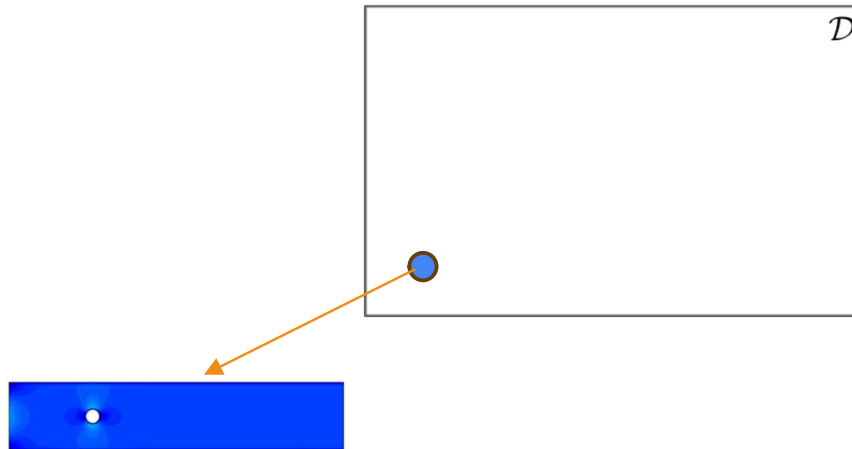
(Parametric Exploration)

$$\mathbf{A}^u = \mathbf{u}_1(\mu_1)$$



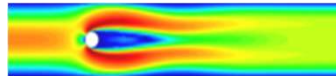
Time dependent (Parametric Exploration)

$$\mathbf{A}^u = \mathbf{u}_1(\boldsymbol{\mu}_1), \mathbf{u}_2(\boldsymbol{\mu}_1), \dots, \mathbf{u}_T(\boldsymbol{\mu}_1)$$



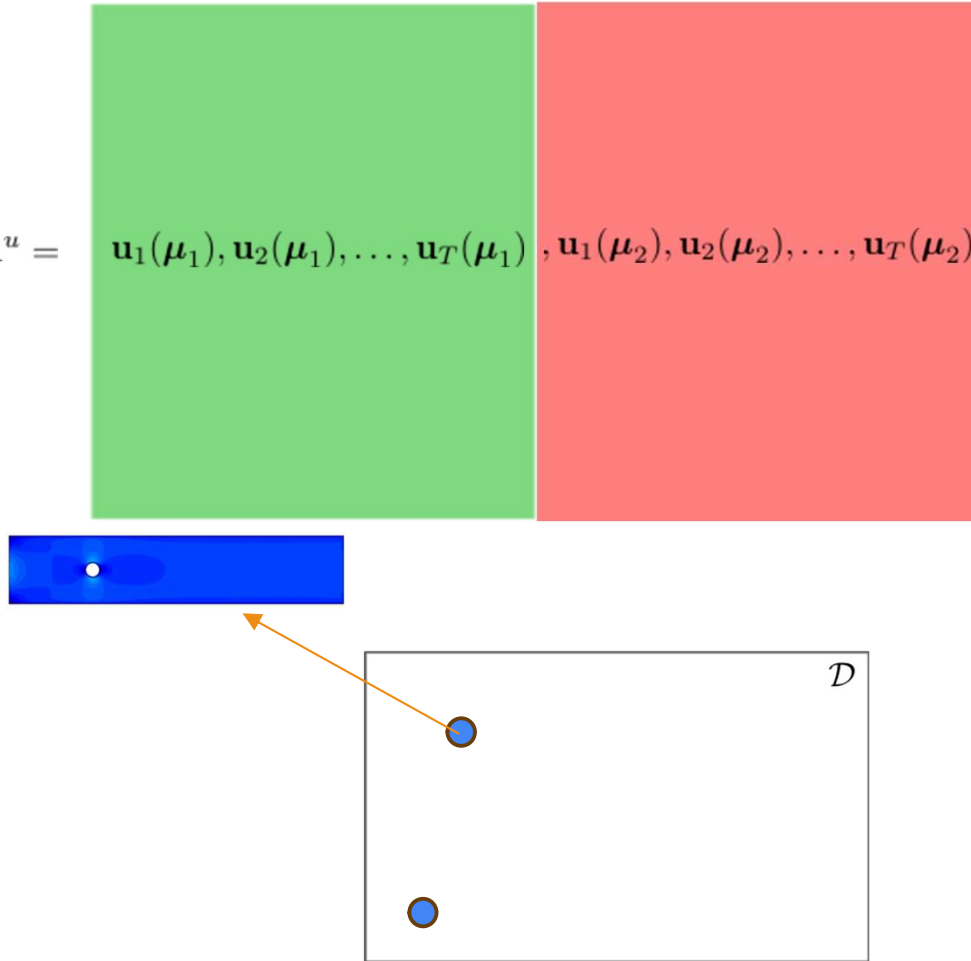
Time dependent (Parametric Exploration)

$$\mathbf{A}^u = \mathbf{u}_1(\mu_1), \mathbf{u}_2(\mu_1), \dots, \mathbf{u}_T(\mu_1), \mathbf{u}_1(\mu_2)$$



Time dependent (Parametric Exploration)

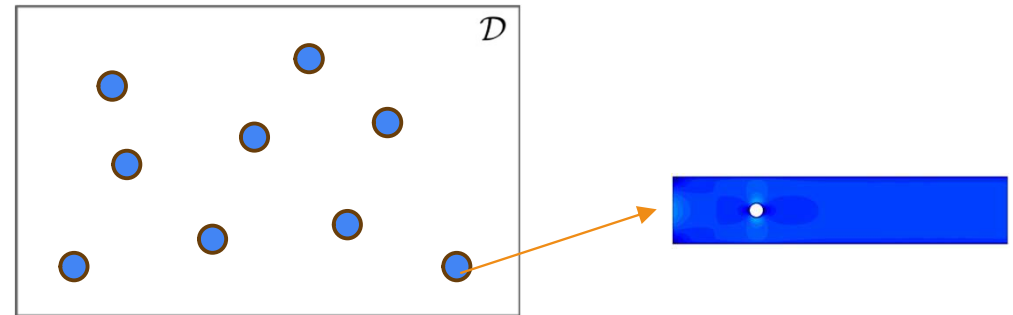
$$\mathbf{A}^u = \mathbf{u}_1(\mu_1), \mathbf{u}_2(\mu_1), \dots, \mathbf{u}_T(\mu_1) , \mathbf{u}_1(\mu_2), \mathbf{u}_2(\mu_2), \dots, \mathbf{u}_T(\mu_2)$$



Time dependent

(Parametric Exploration)

$$\mathbf{A}^u = \mathbf{u}_1(\mu_1), \mathbf{u}_2(\mu_1), \dots, \mathbf{u}_T(\mu_1) , \mathbf{u}_1(\mu_2), \mathbf{u}_2(\mu_2), \dots, \mathbf{u}_T(\mu_2) , \dots, \mathbf{u}_T(\mu_P)$$



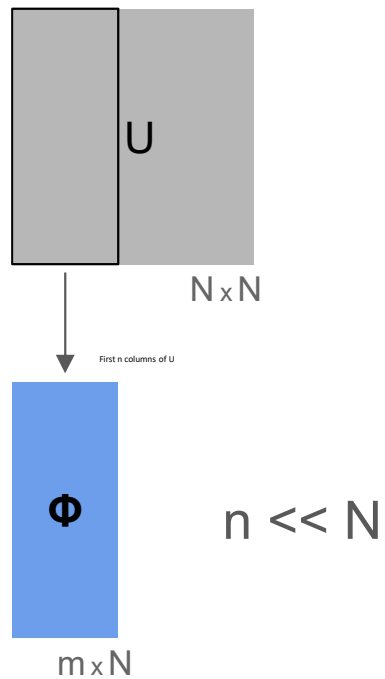
Time dependent

(Parametric Exploration)

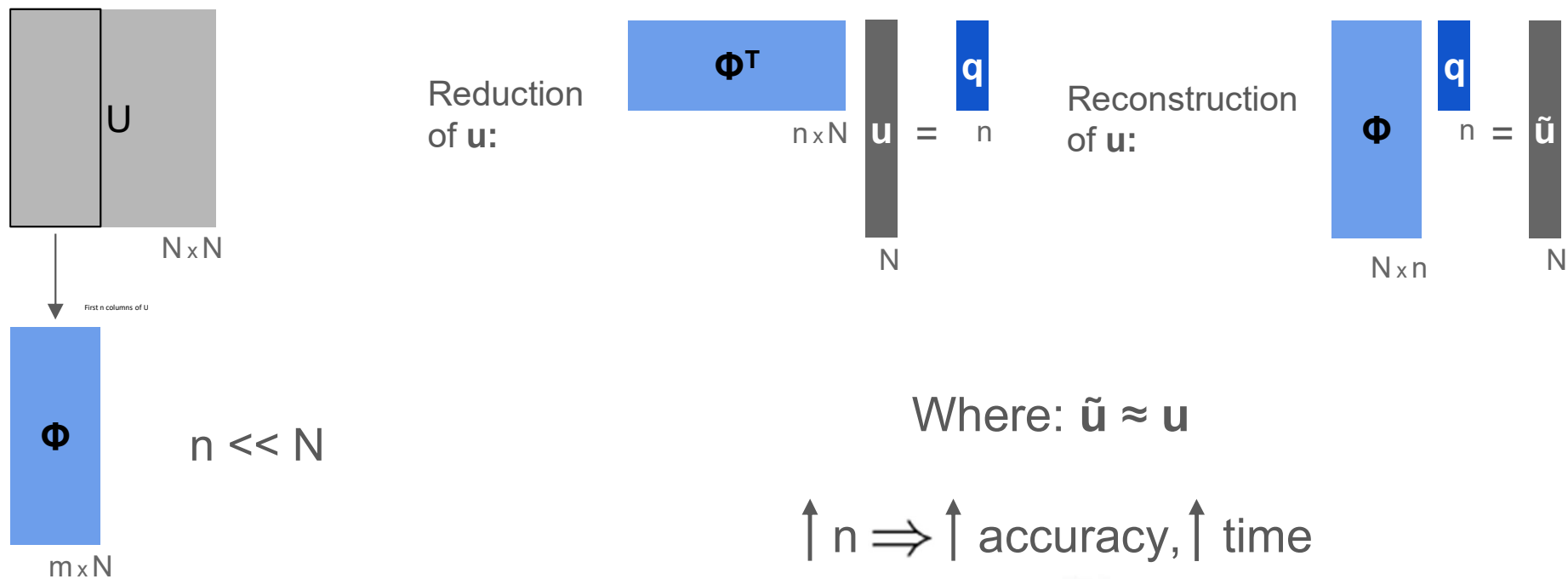
$$\mathbf{A}^u = [\mathbf{u}_1(\mu_1), \mathbf{u}_2(\mu_1), \dots, \mathbf{u}_T(\mu_1), \mathbf{u}_1(\mu_2), \mathbf{u}_2(\mu_2), \dots, \mathbf{u}_T(\mu_2), \dots, \mathbf{u}_T(\mu_P)]$$

$N \times (T * P)$

Use modes in U as a base to project the full system into a smaller dimension



Use modes in U as a base to project the full system into a smaller dimension



ONLINE:

Original system:

$$\begin{array}{ccc} \text{J} & \mathbf{u} & = \mathbf{R} \\ N \times N & N & N \end{array}$$

We use our basis Φ to project the problem from dimension N to dimension n ($n \ll N$)

$$\begin{array}{ccccc} \Phi^T & \text{J} & \Phi & \mathbf{q} & = \\ n \times N & N \times N & N \times n & n & \\ & & & & \Phi^T \quad \mathbf{R} \\ & & & & n \times N \quad N \end{array}$$

System to solve:

$$\begin{array}{ccc} & \mathbf{q} & = \\ n \times n & n & n \end{array}$$



Further speedup: HROM

$$\begin{matrix} \Phi^T \mathbf{J}_t^{(k)} \Phi & \delta \mathbf{q}_t^k & = - & \Phi^T \mathbf{R}_t^{(k)} \\ n \times n & n & & n \end{matrix}$$

ROM Assembling:



$$\Phi^T \mathbf{R} = \sum_{e=1}^L \Phi_e^T \mathbf{R}_e$$

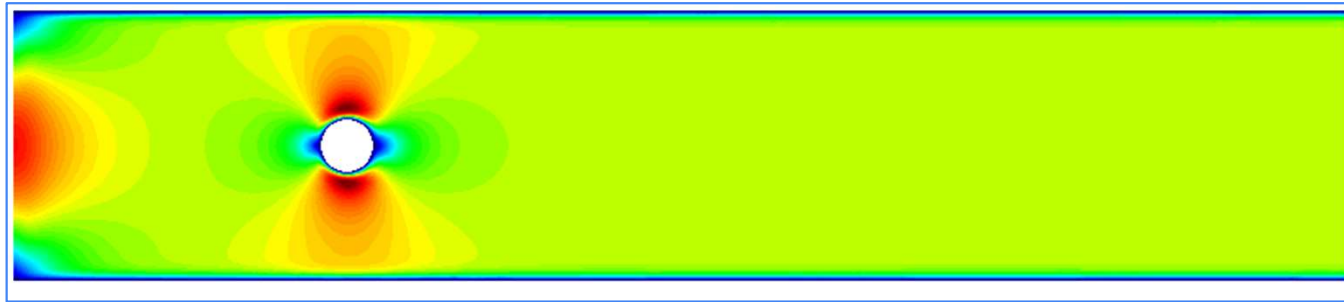
HROM Assembling:



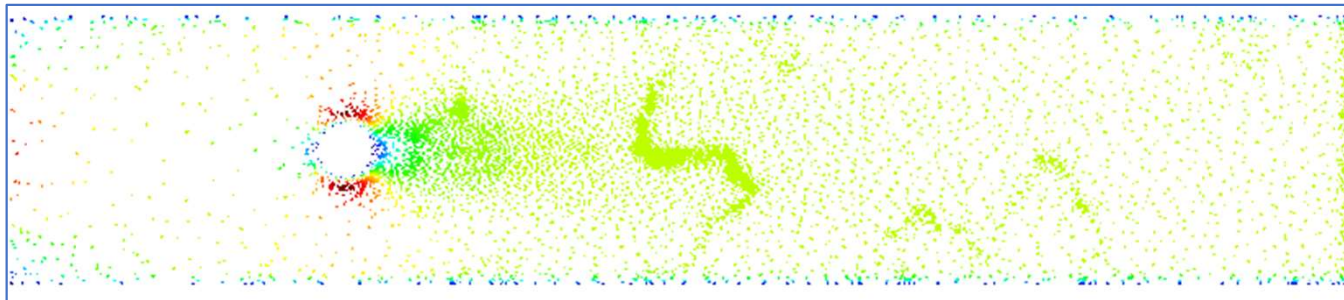
$$\Phi^T \mathbf{R} \approx \sum_{e \in S} w_e \Phi_e^T \mathbf{R}_e$$

22

ROM vs HROM



68298 elements – Speed up: 4.04



6216 elements – Speed up: 49

SVD projection is the **best linear** approximation

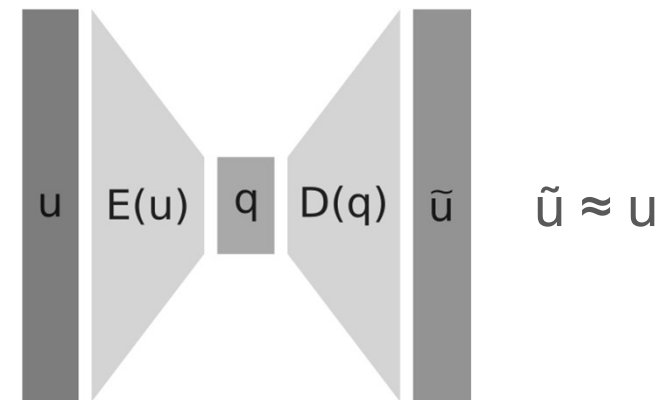
What if it is not enough for our case?

→ We have to go with **non-linear** techniques

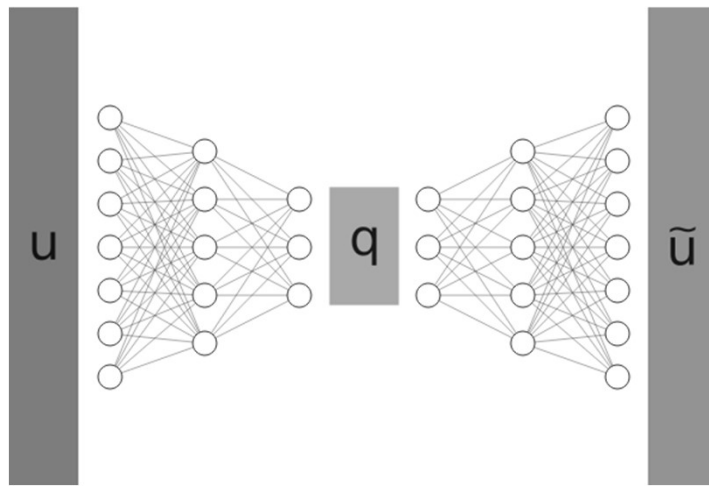
In a generic way, we will always have
an encoding-decoding routine

$$\begin{aligned} \text{POD: } E(u) &= \Phi^T u \\ D(q) &= \Phi q \end{aligned}$$

Which operators to choose?



First guess: Neural Networks



- Very generalizable
- Simple and efficient to run once trained

- Scalability problems
- Long training process

Best bet in this sense are Graph Neural Networks (future work)



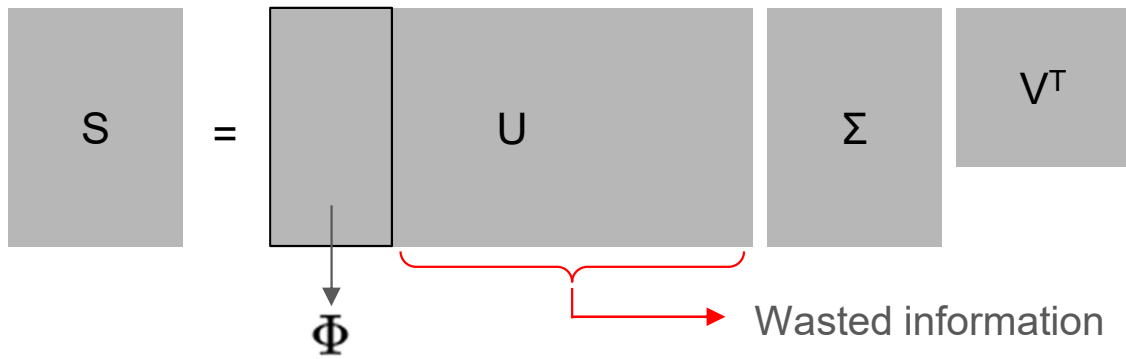
Other alternatives? **ANN-PROM architecture**

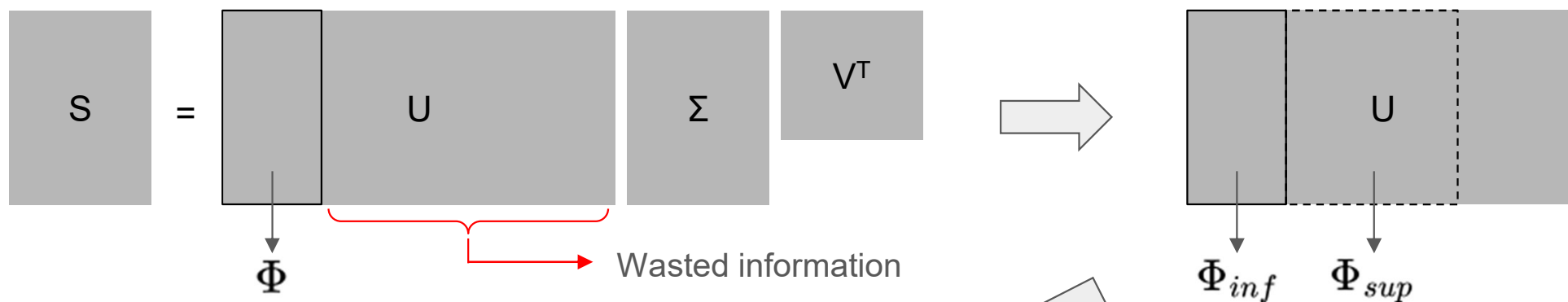
First proposed in:

[1] J. Barnett, C. Farhat, and Y. Maday. Neural-network-augmented projection-based model order reduction for mitigating the kolmogorov barrier to reducibility. Journal of Computational Physics, 492:112420, 2023.

Use Neural Networks to build on top of the basic POD approach



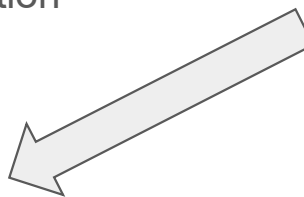
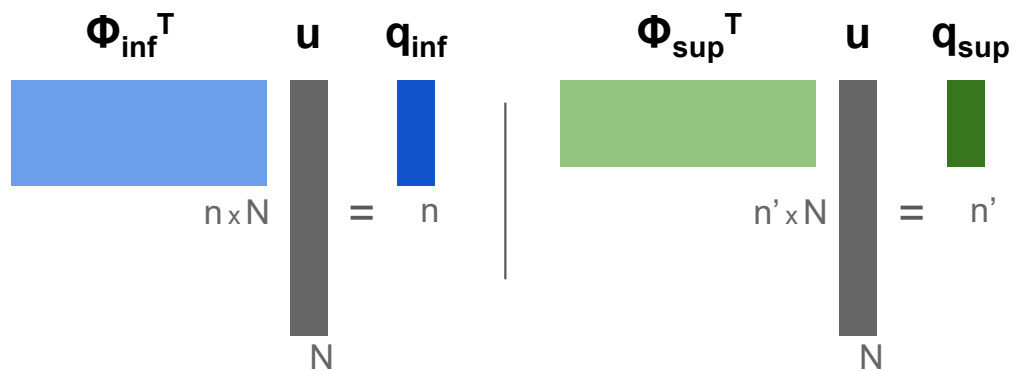
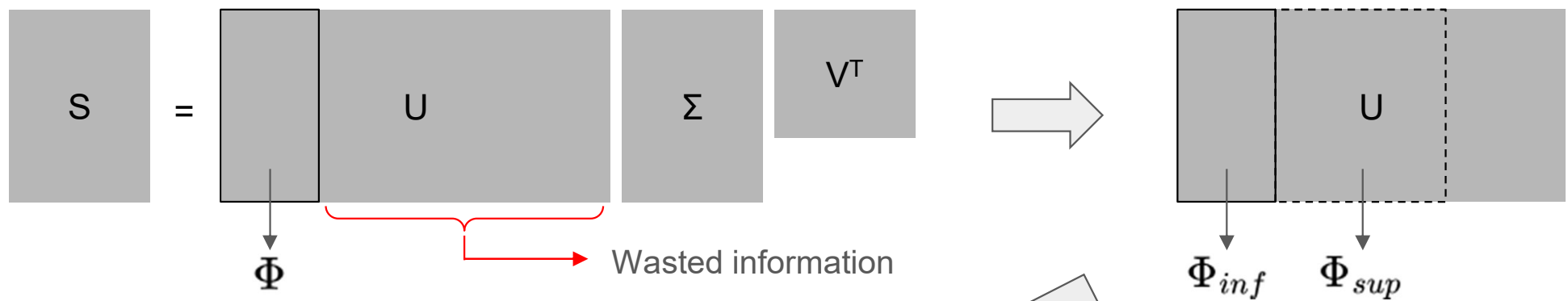




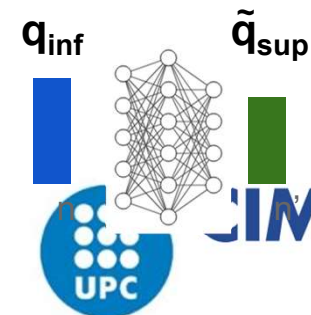
$$\begin{matrix} \Phi_{inf}^T & u & q_{inf} \\ \text{[Blue Block]} & \text{[Gray Block]} & \text{[Blue Block]} \\ n \times N & = & n \\ & N & \end{matrix}$$

$$\begin{matrix} \Phi_{sup}^T & u & q_{sup} \\ \text{[Green Block]} & \text{[Gray Block]} & \text{[Green Block]} \\ n' \times N & = & n' \\ & N & \end{matrix}$$





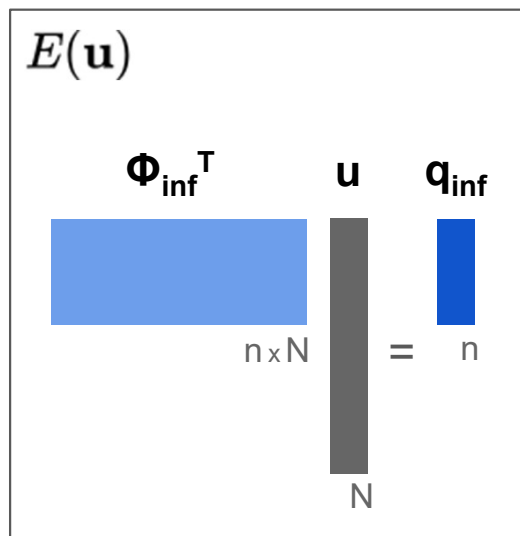
$$q_{sup} \approx N(q_{inf})$$



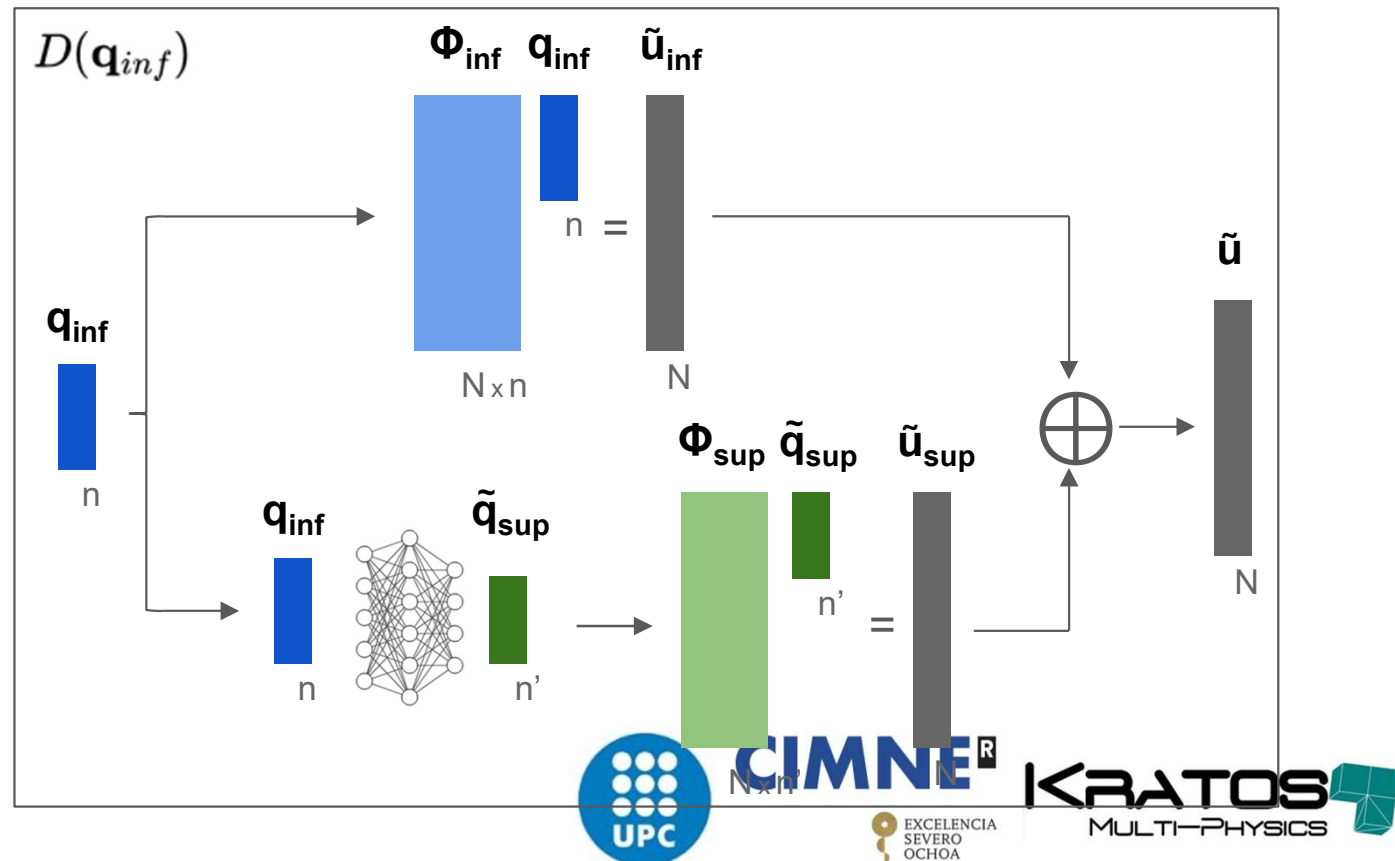
By using the NN on the modes instead of the DoFs we avoid scalability issues

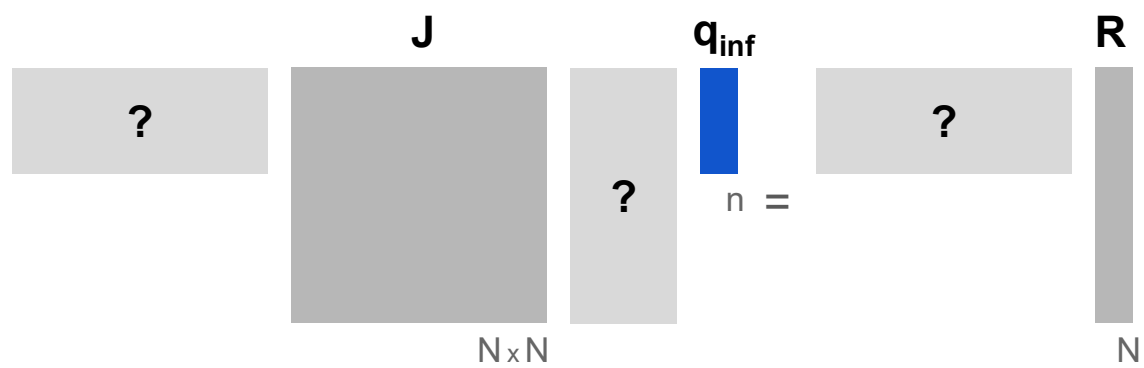


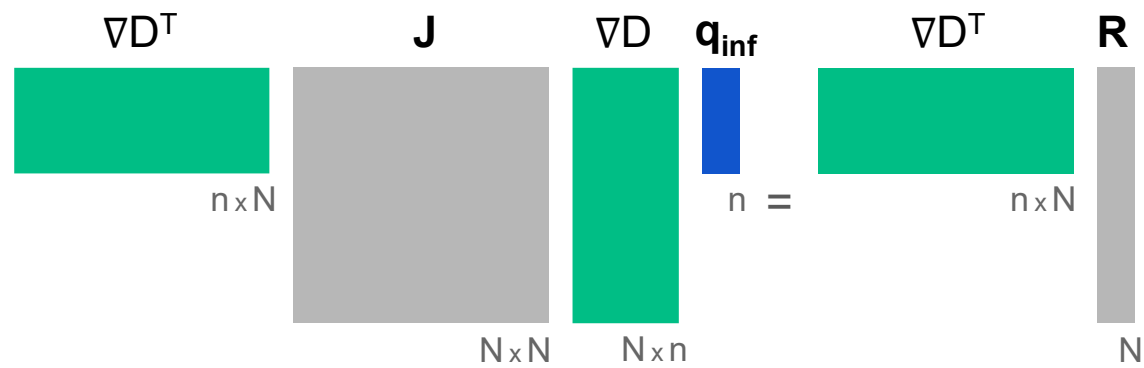
$$E(\mathbf{u}) = \Phi_{inf}^T \mathbf{u} = \mathbf{q}_{inf}$$



$$D(\mathbf{q}_{inf}) = \Phi_{inf} \mathbf{q}_{inf} + \Phi_{sup} N(\mathbf{q}_{inf}) = \tilde{\mathbf{u}}$$



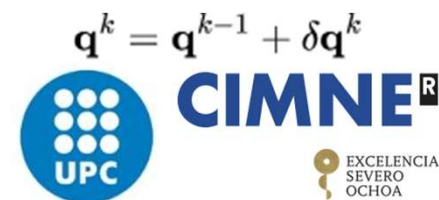




$$\nabla D := \left. \frac{\partial D}{\partial \mathbf{q}} \right|_{\mathbf{q}^{k-1}} = \Phi_{inf} + \Phi_{sup} \left. \frac{\partial N}{\partial \mathbf{q}} \right|_{\mathbf{q}^{k-1}}$$

Formally:

$$\left. \frac{\partial D}{\partial \mathbf{q}} \right|_{\mathbf{q}^{k-1}}^T \left. \frac{\partial \mathbf{R}}{\partial \mathbf{u}} \right|_{D(\mathbf{q}^{k-1}); \mu} \frac{\partial D}{\partial \mathbf{q}} \bigg|_{\mathbf{q}^{k-1}} \delta \mathbf{q}^k = - \left. \frac{\partial D}{\partial \mathbf{q}} \right|_{\mathbf{q}^{k-1}}^T \mathbf{R}(D(\mathbf{q}^{k-1}); \mu)$$



Let's look at the NN's training loss

- Farhat's: Mean Squared Error directly over **reduced snapshot**

$$\mathcal{L}_{Farhat} = \frac{1}{m_b n} \sum_{j=1}^{m_b} \|N(\Phi_{inf}^T \mathbf{u}_j) - \Phi_{sup}^T \mathbf{u}_j\|^2 = \frac{1}{m_b n} \sum_{j=1}^{m_b} \|\tilde{\mathbf{q}}_{sup}^* - \mathbf{q}_{sup}^*\|^2$$

No information on which modes are the most important for each case

- Ours: MSE over **full snapshot**

$$\mathcal{L}_D = \frac{1}{e_{base,D}} \frac{1}{m_b N} \sum_{j=1}^{m_b} \|D(E(\mathbf{u}_j)) - \mathbf{u}_j\|^2 = \frac{1}{e_{base,D}} \frac{1}{m_b N} \sum_{j=1}^{m_b} \|\tilde{\mathbf{u}}_j - \mathbf{u}_j\|^2$$

Gradient of the loss implicitly uses Φ_{sup} to indicate importance of modes



What about $e_{base,D}$?

$$\mathcal{L}_D = \underbrace{\frac{1}{e_{base,D}}}_{\text{Constant factor}} \frac{1}{m_b N} \sum_{j=1}^{m_b} \|D(E(\mathbf{u}_j)) - \mathbf{u}_j\|^2 = \frac{1}{e_{base,D}} \frac{1}{m_b N} \sum_{j=1}^{m_b} \|\tilde{\mathbf{u}}_j - \mathbf{u}_j\|^2$$
$$e_{base,D} = \frac{1}{mN} \sum_{j=1}^m \|\Phi_{inf} \Phi_{inf}^T \mathbf{u}_j - \mathbf{u}_j\|^2$$

MSE for the error of the POD over all training snapshots (with n_{inf} modes)

- Easily see how much better our model is compared to simple POD
- Makes loss independent of number of inferior modes

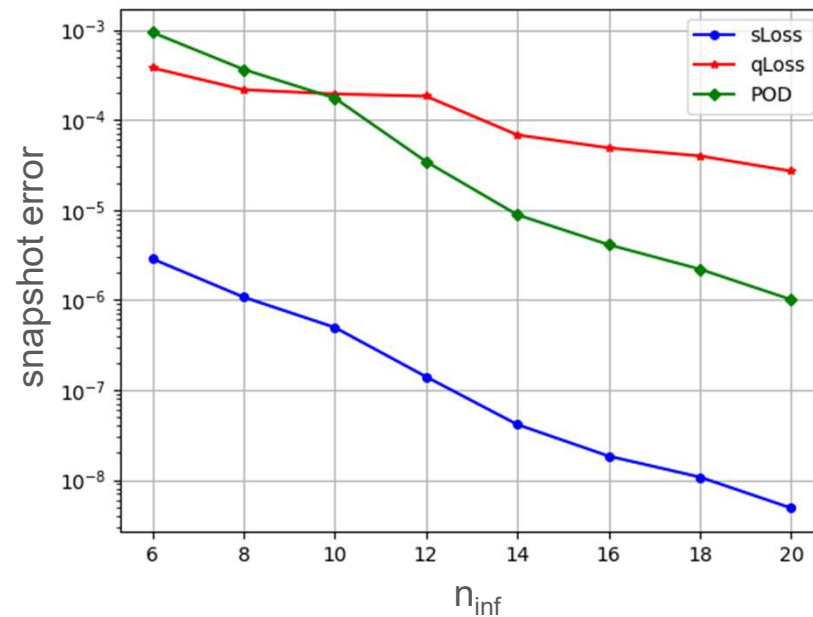


Reconstruction error ($\|D(E(\mathbf{S})) - \mathbf{S}\|_F$) for
different methods:

Varying n_{inf}

$$n_{\text{sup}} = 60 - n_{\text{inf}}$$

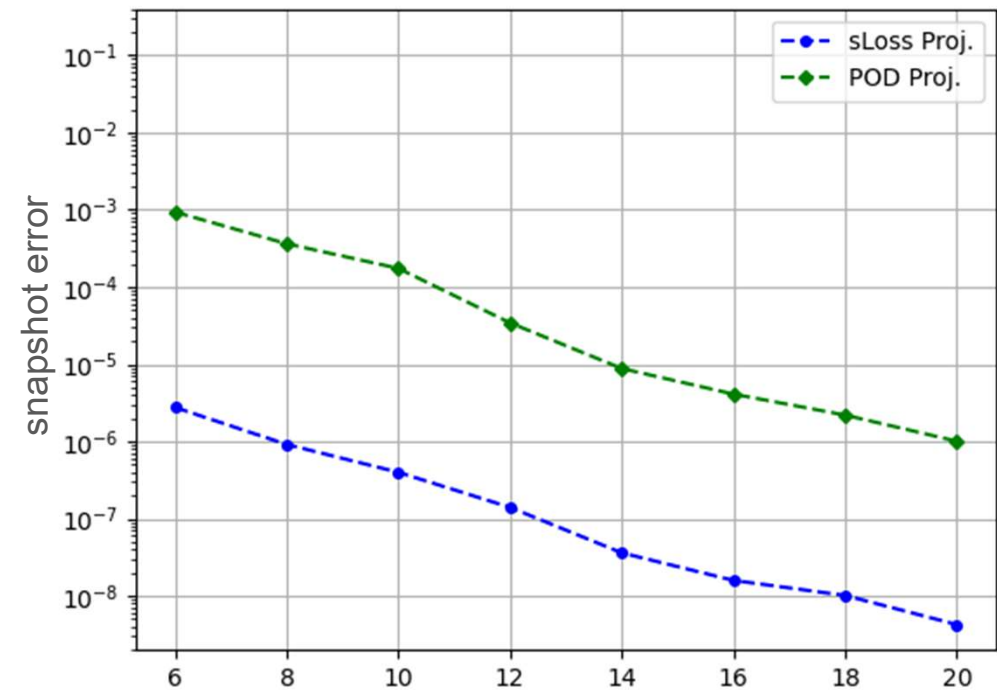
- POD
- ANN-PROM (original loss)
- ANN-PROM (full snapshot loss)



Reconstruction error:

$$\|D(E(\mathbf{S})) - \mathbf{S}\|_F$$

- POD
- ANN-PROM (full snapshot loss)



Reconstruction error:

$$\|D(E(\mathbf{S})) - \mathbf{S}\|_F$$

-- POD

-- ANN-PROM (full snapshot loss)

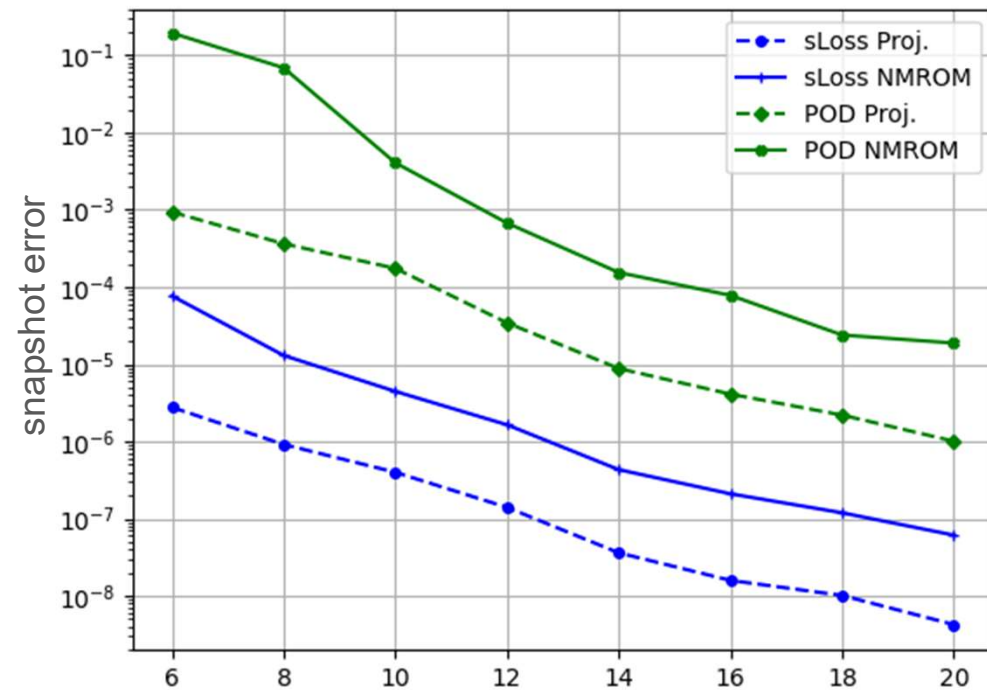
VS

Online error:

$$\|\mathbf{S}_{ROM} - \mathbf{S}\|_F$$

— POD

— ANN-PROM (full snapshot loss)



We design an alternative loss, based on the error in the snapshot's **residual**.

$$\mathcal{L}_R = \frac{1}{e_{base,R}} \frac{1}{m_b N} \sum_{j=1}^{m_b} \|\mathbf{R}(D(E(\mathbf{u}_j)); \boldsymbol{\mu}) - \mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu})\|^2$$

$$e_{base,R} = \frac{1}{mN} \sum_{j=1}^m \|\mathbf{R}(\Phi_{inf} \Phi_{inf}^T \mathbf{u}_j; \boldsymbol{\mu}) - \mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu})\|^2$$



Implementation of this loss requires interaction Kratos - TensorFlow

$$\mathcal{L}_R = \frac{1}{e_{base,R}} \frac{1}{m_b N} \sum_{j=1}^{m_b} \left\| \underbrace{\mathbf{R}(D(E(\mathbf{u}_j)); \boldsymbol{\mu})}_{\text{Kratos}} - \underbrace{\mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu})}_{\text{Kratos}} \right\|^2$$

$$\frac{\partial \mathcal{L}_R}{\partial \boldsymbol{\theta}} = \left[\frac{\partial \mathcal{L}_R}{\partial \mathbf{R}} \right] \left[\frac{\partial \mathbf{R}}{\partial \mathbf{u}} \right] \left[\frac{\partial \mathbf{u}}{\partial \boldsymbol{\theta}} \right]$$

TF/Pytorch Kratos TF/Pytorch

$$\frac{\partial \mathcal{L}_R}{\partial \boldsymbol{\theta}} = \frac{1}{e_{base,R}} \frac{2}{m_b N} \sum_{j=1}^m \underbrace{(\mathbf{R}(D(E(\mathbf{u}_i))) - \mathbf{R}(\mathbf{u}_i))^T}_{\text{Must be done one by one in Kratos}} \underbrace{\frac{\partial \mathbf{R}}{\partial \mathbf{u}} \frac{\partial D(E(\mathbf{u}_i))}{\partial \boldsymbol{\theta}}}_{\text{Evaluating each jacobian individually in TF is expensive}}$$

Must be done one by one
in Kratos

Evaluating each jacobian
individually in TF is expensive



Instead, pre-compute the following vector for all samples and save them:

$$\mathbf{v}_i^T := \frac{1}{e_{base,R}} \frac{2}{m_b N} (\mathbf{R}(D(E(\mathbf{u}_i))) - \mathbf{R}(\mathbf{u}_i))^T \frac{\partial \mathbf{R}}{\partial \mathbf{u}}$$

Then re-write the loss as if these were constant vectors:

$$\frac{\partial \mathcal{L}_R}{\partial \theta} = \sum_{i=1}^{m_b} \boxed{\mathbf{v}_i^T} \frac{\partial D(E(\mathbf{u}_i))}{\partial \theta} = \frac{\partial}{\partial \theta} \left(\sum_{i=1}^{m_b} \boxed{\mathbf{v}_i^T} \cdot D(E(\mathbf{u}_i)) \right)$$

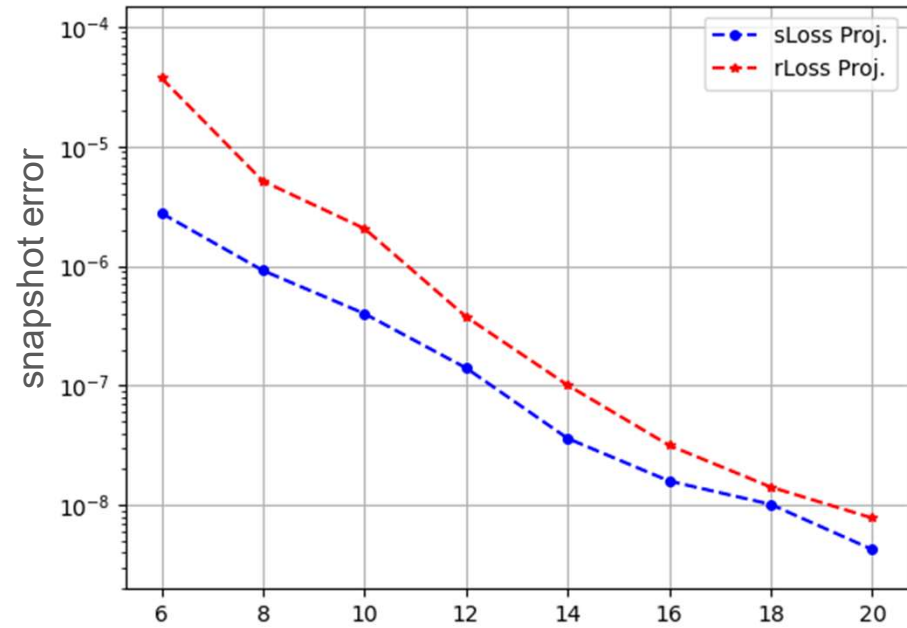
↑ Treated as constants ↑

Now we are calculating a single gradient for a scalar loss → Much more efficient



Reconstruction error: $\|D(E(\mathbf{S})) - \mathbf{S}\|_F$

- Residual loss
- Snapshot loss

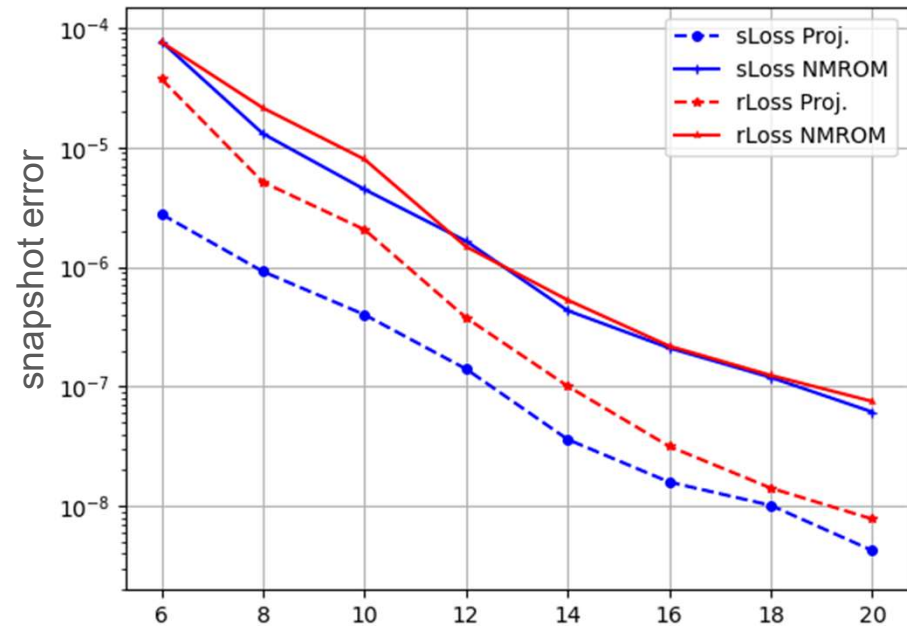


Reconstruction error: $\|D(E(\mathbf{S})) - \mathbf{S}\|_F$

-- Residual loss
-- Snapshot loss

ROM error: $\|\mathbf{S}_{ROM} - \mathbf{S}\|_F$

— Residual loss
— Snapshot loss

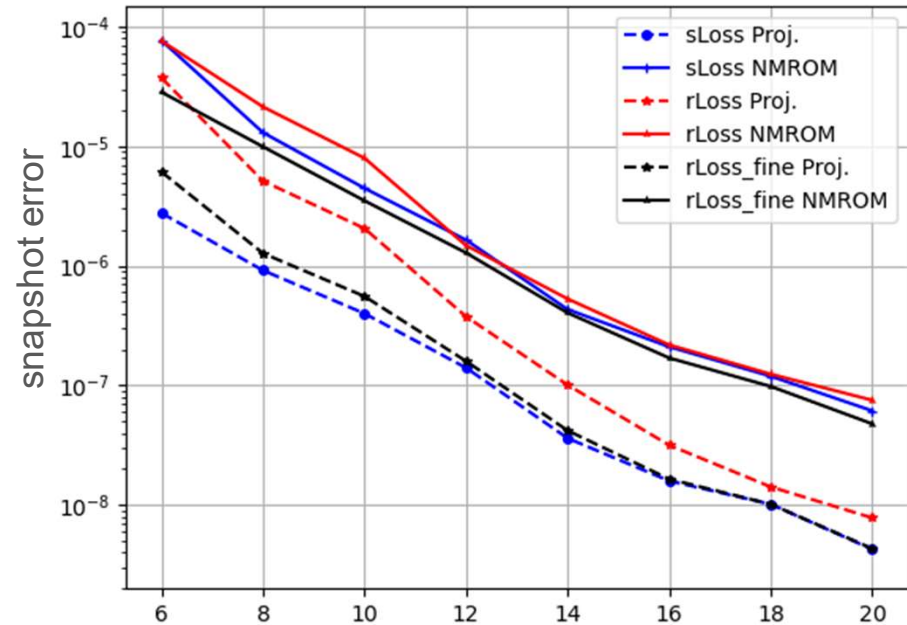


Reconstruction error: $\|D(E(\mathbf{S})) - \mathbf{S}\|_F$

- Residual loss
- Snapshot loss
- Finetuned (snapshot + residual loss)

ROM error: $\|\mathbf{S}_{ROM} - \mathbf{S}\|_F$

- Residual loss
- Snapshot loss
- Finetuned (snapshot + residual loss)



Pros:

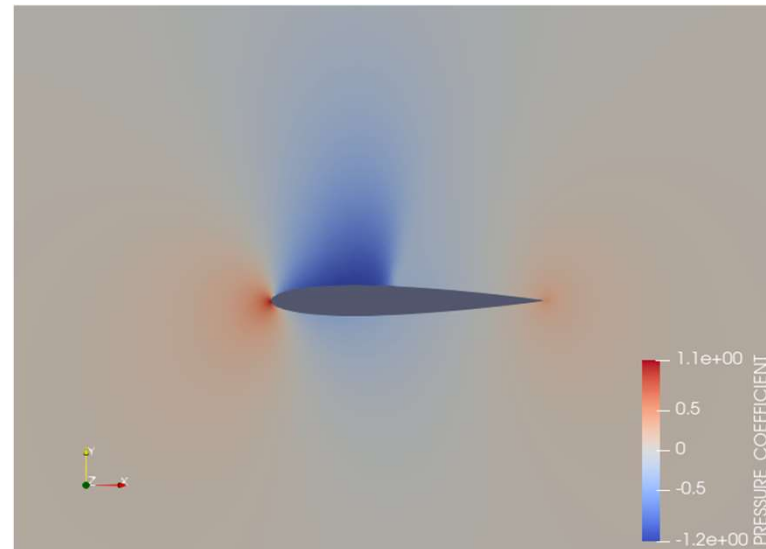
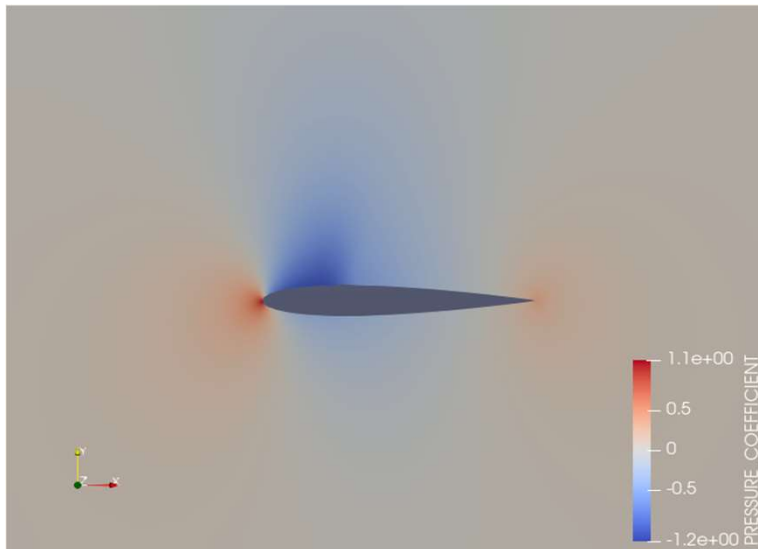
- Very simple to understand and implement
- Scalability problems are heavily relieved
- Good performance when trained with snapshot loss

Cons:

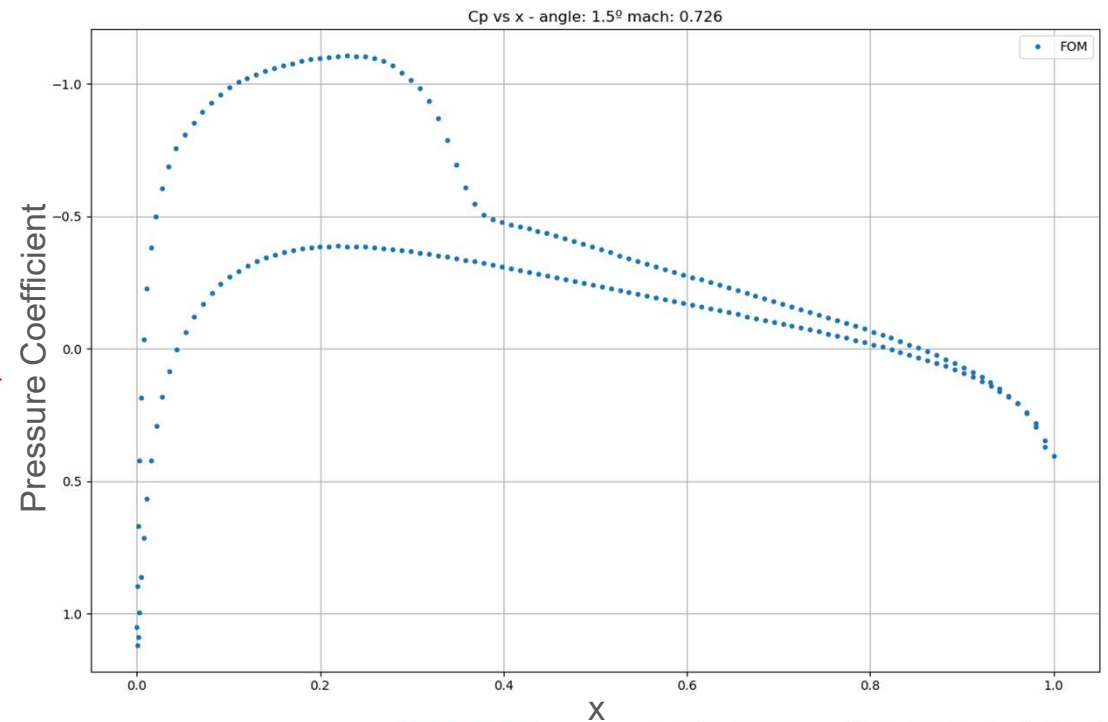
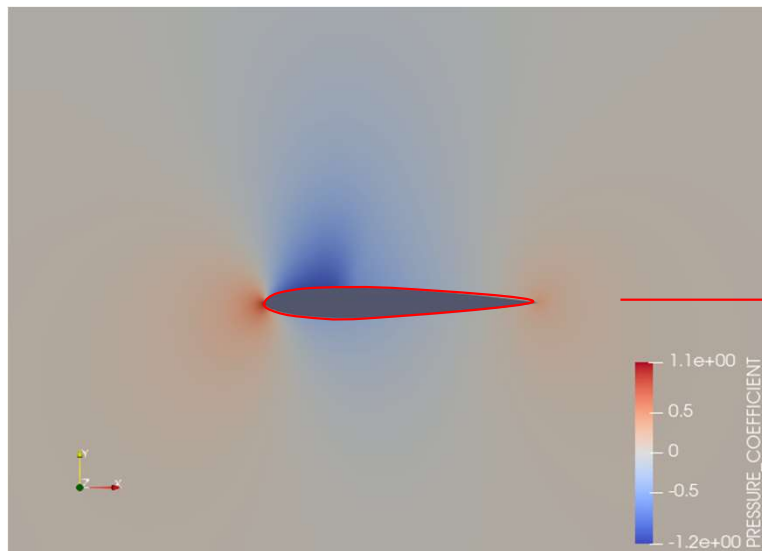
- Can't go further than POD for $n_{\text{inf}} + n_{\text{sup}}$ modes
- Requires more training data than traditional POD
- Residual loss is too difficult to train



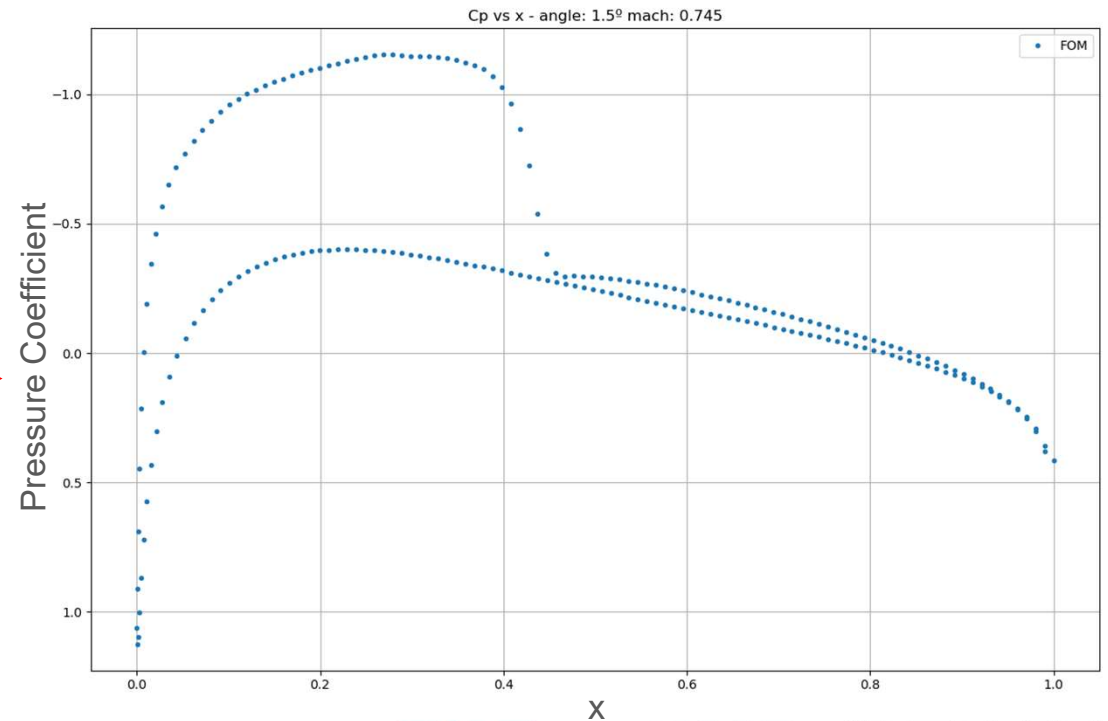
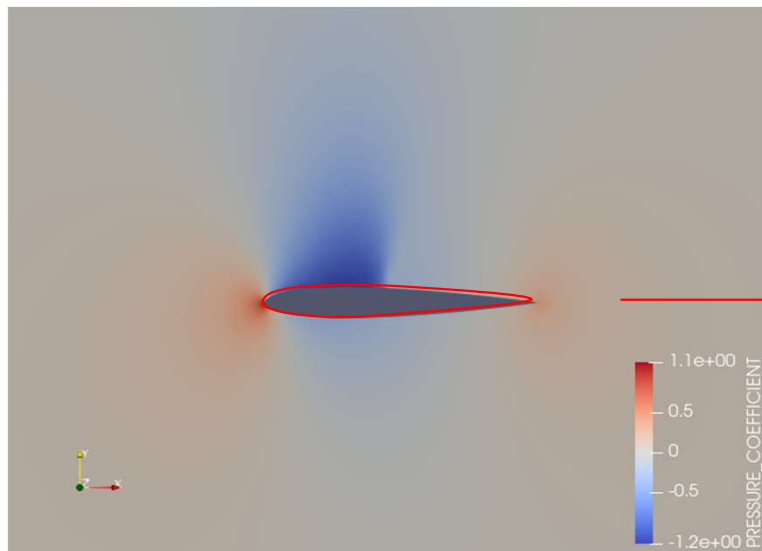
Compressible potential flow in transonic regime,
over a wing profile



Goal: Find shock points through the pressure coefficient over wing surface



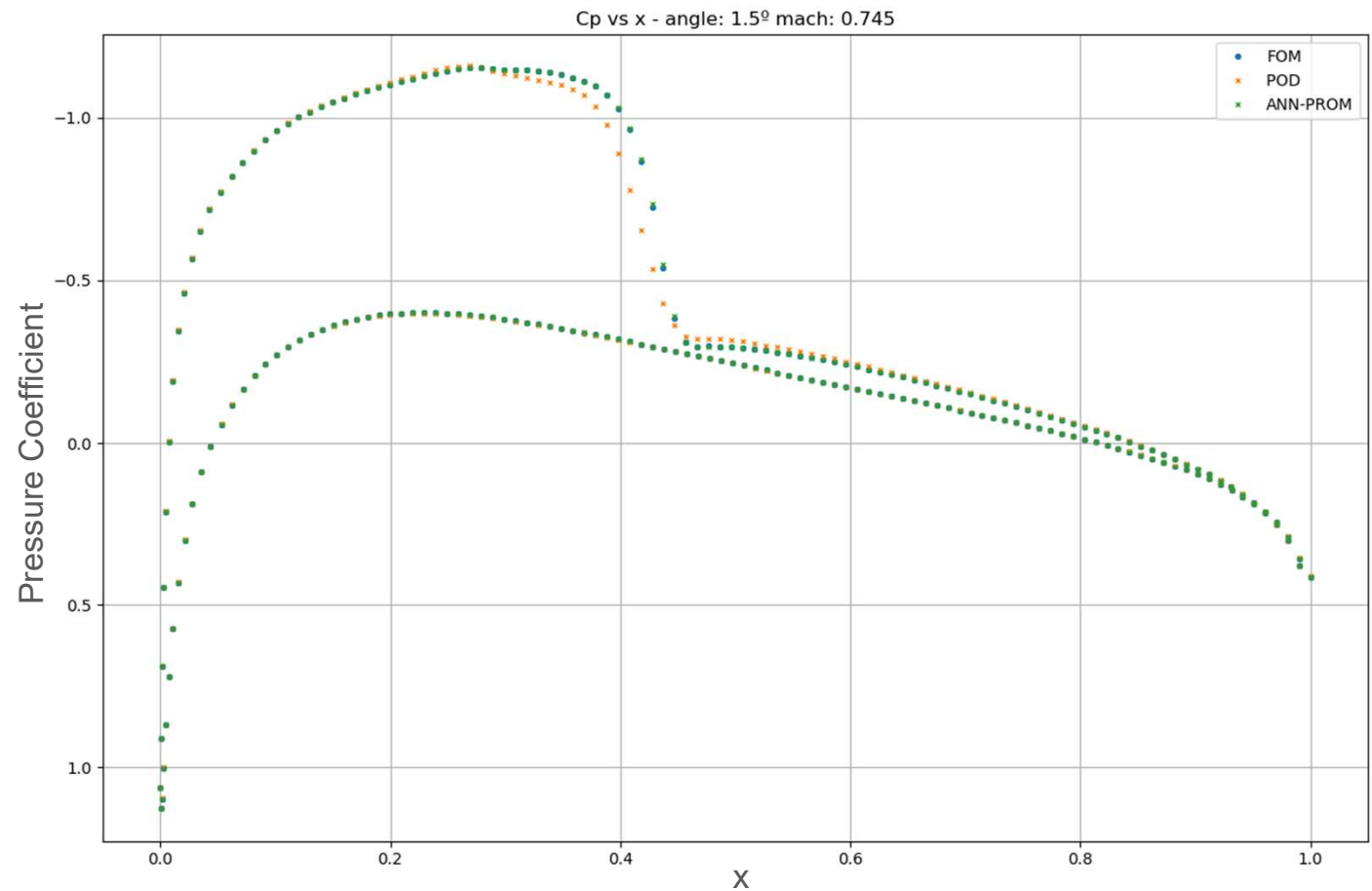
Goal: Find shock points through the pressure coefficient over wing surface



Mach: 0.745

Comparison:

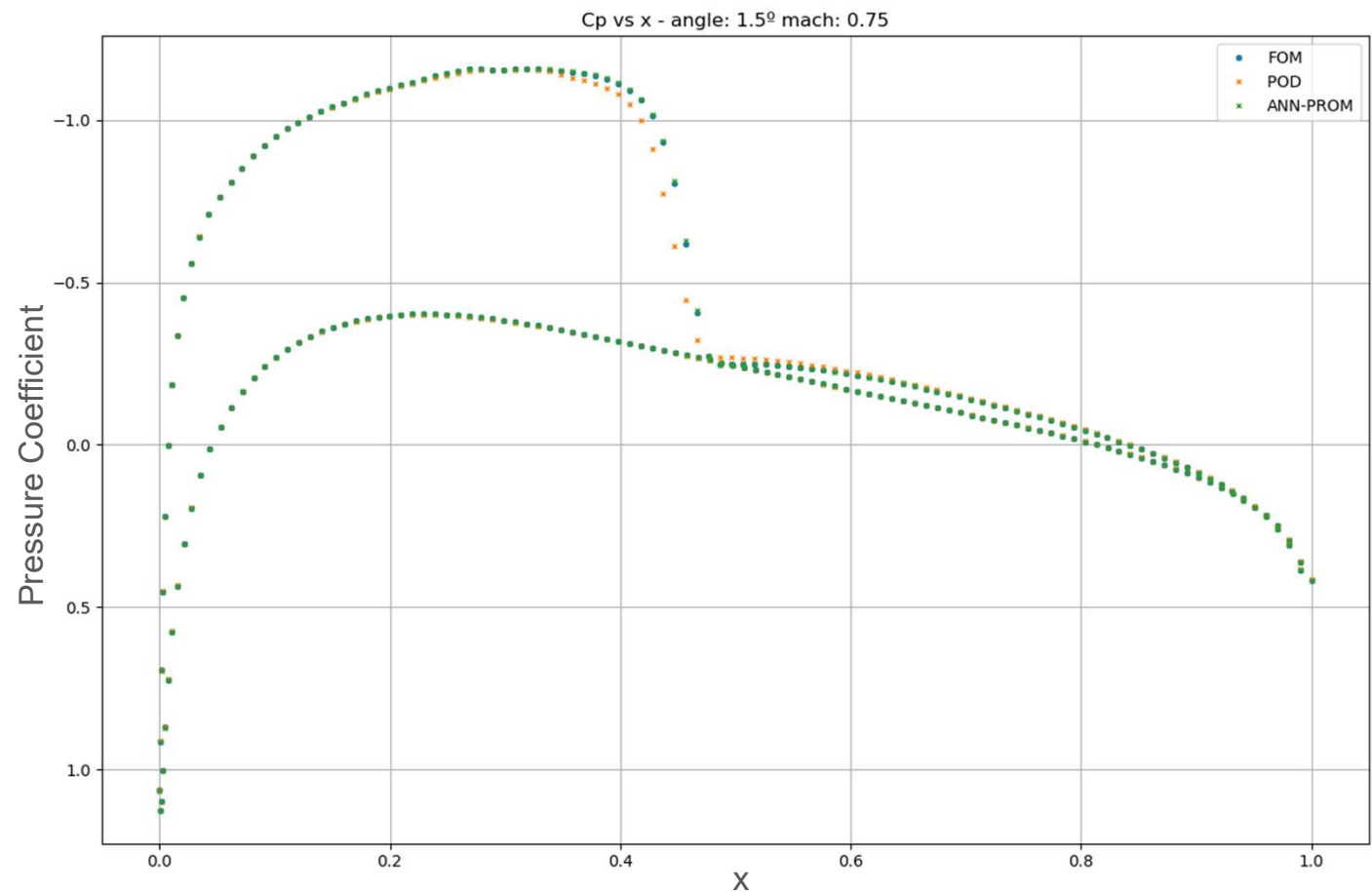
- FOM
- POD (5 modes)
- ANN-PROM (5 inf. modes, 465 sup. modes)



Mach: 0.75

Comparison:

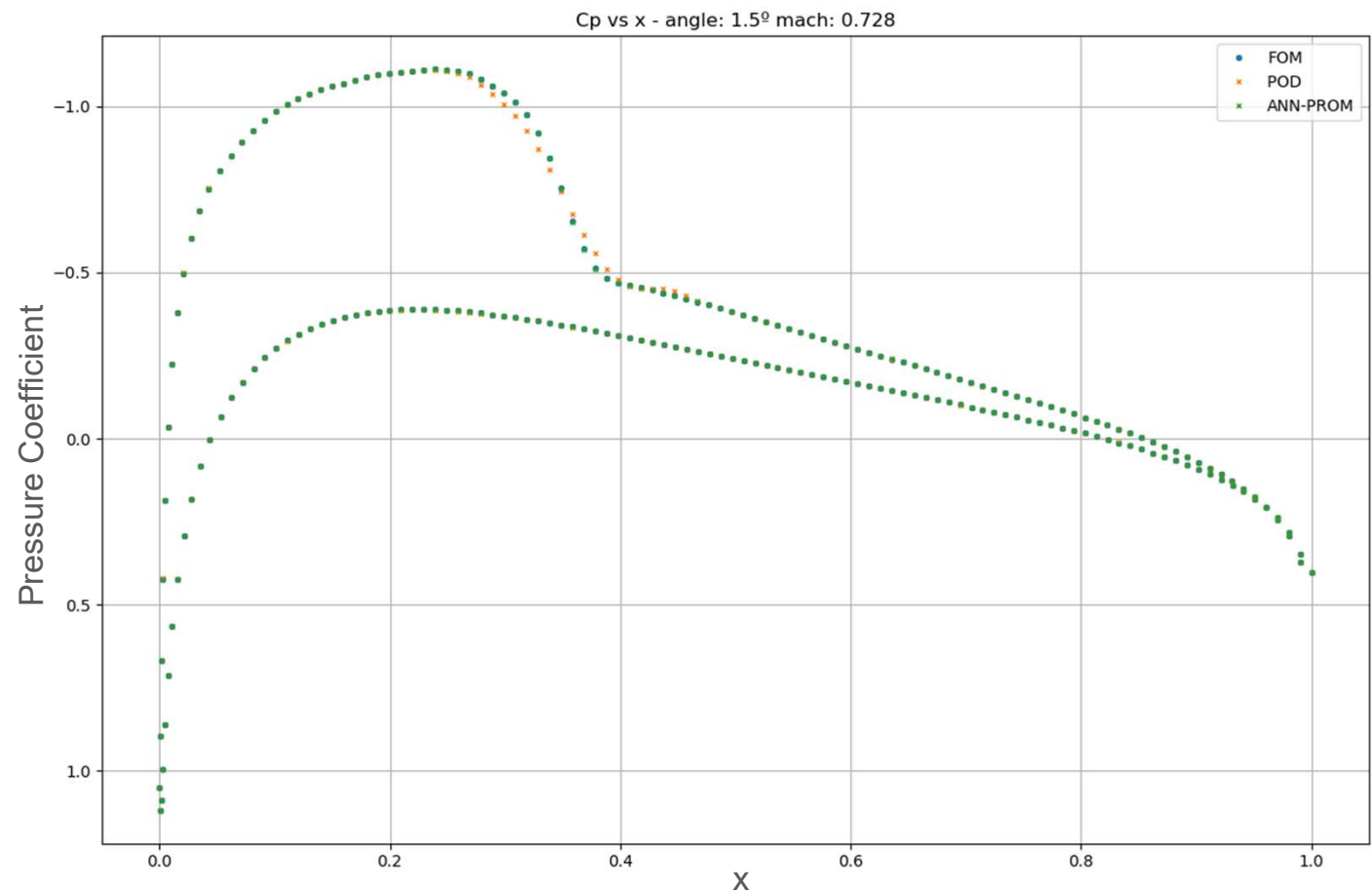
- FOM
- POD (5 modes)
- ANN-PROM (5 inf. modes, 465 sup. modes)



Mach: 0.728

Comparison:

- FOM
- POD (5 modes)
- ANN-PROM (5 inf. modes, 465 sup. modes)



Issues in this use case

- Not able to train the residual (many local minima in the loss function space)
- If varying both mach and angle of attack, the sharpest shocks are not captured properly. More modes and training data are needed around those points in the parametric space



In order to improve, these issues should be tackled

- Parametric space exploration and sampling
- Selection of inferior and superior modes
- Finding methods (or alternative architectures) which make the residual loss more stable



ANN-PROM is openly available through the ROM application

- Training routine is implemented in TensorFlow
 - Only Snapshot loss is available
 - HROM not implemented yet
- Online stage is independent of TensorFlow



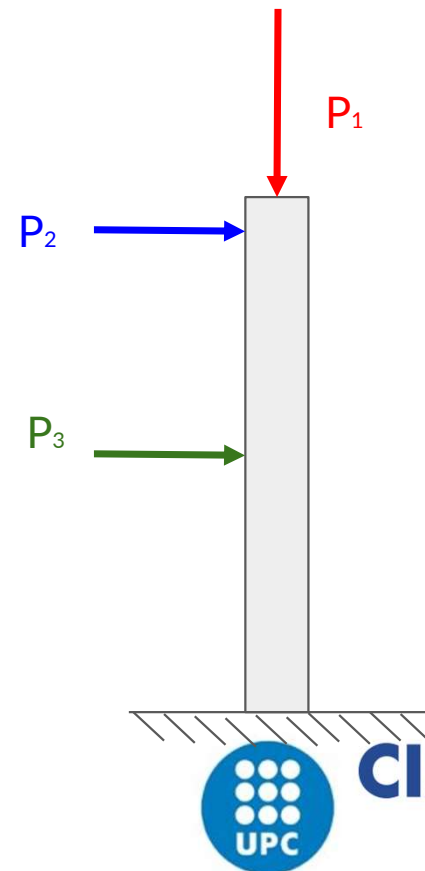
What can be done now (DEMO)?

$$\begin{cases} \mathcal{R}(\mathbf{u}(\boldsymbol{\mu})) = \mathbf{0} & \text{in } \Omega(\boldsymbol{\mu}) \\ (\mathbf{u}_D) = g_D(\mathbf{x}, \mathbf{u}) & \text{on } \partial\Omega_D(\boldsymbol{\mu}) \\ (\mathbf{u}_N) = g_N(\mathbf{x}, \mathbf{u}) & \text{on } \partial\Omega_N(\boldsymbol{\mu}) \end{cases}$$

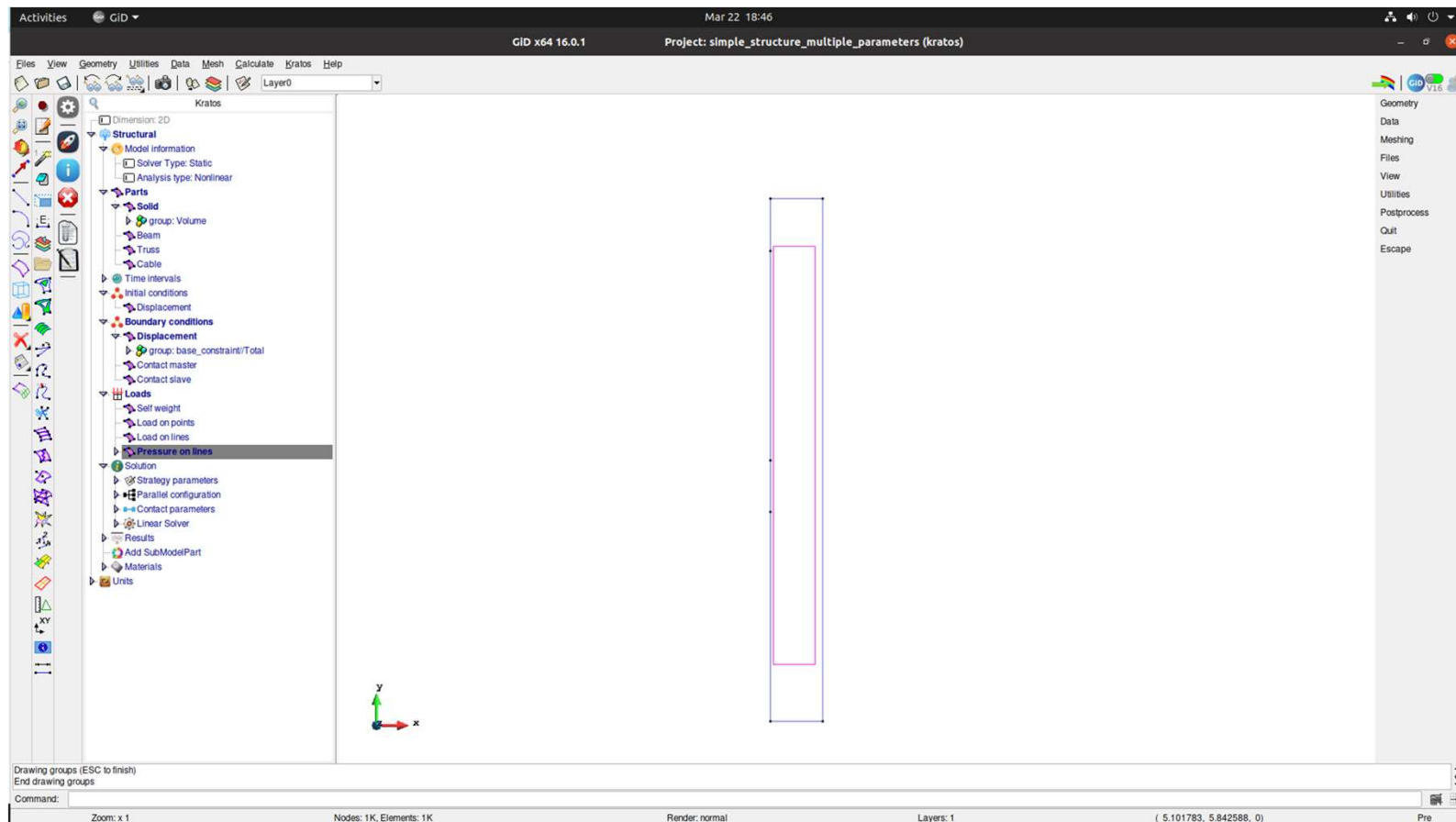
What can be done now (DEMO)?

$$\begin{cases} \mathcal{R}(\mathbf{u}(\boldsymbol{\mu})) = \mathbf{0} & \text{in } \Omega(\boldsymbol{\mu}) \\ (\mathbf{u}_D) = g_D(\mathbf{x}, \mathbf{u}) & \text{on } \partial\Omega_D(\boldsymbol{\mu}) \\ (\mathbf{u}_N) = g_N(\mathbf{x}, \mathbf{u}) & \text{on } \partial\Omega_N(\boldsymbol{\mu}) \end{cases}$$

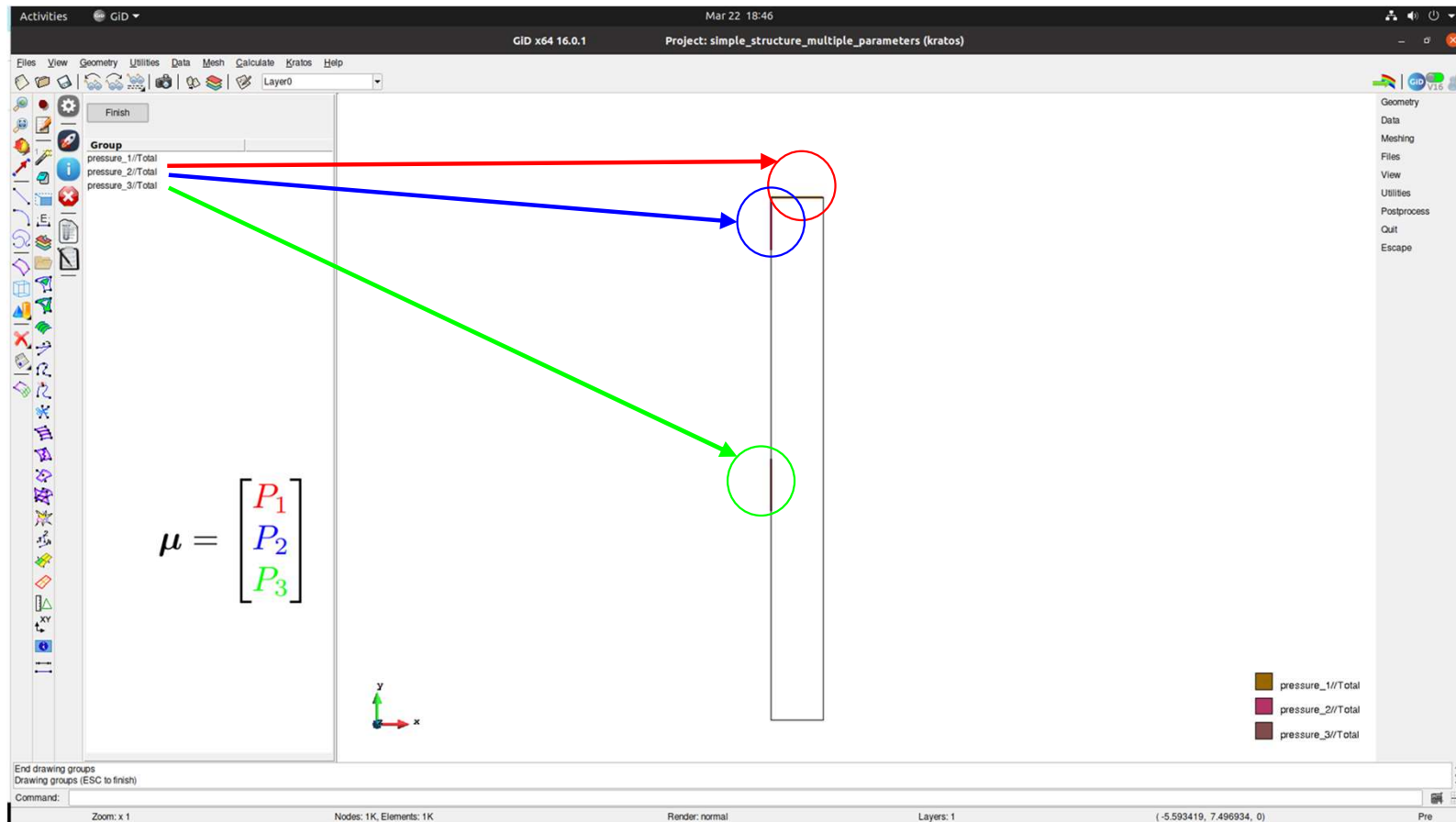
$$\boldsymbol{\mu} = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \end{bmatrix}$$



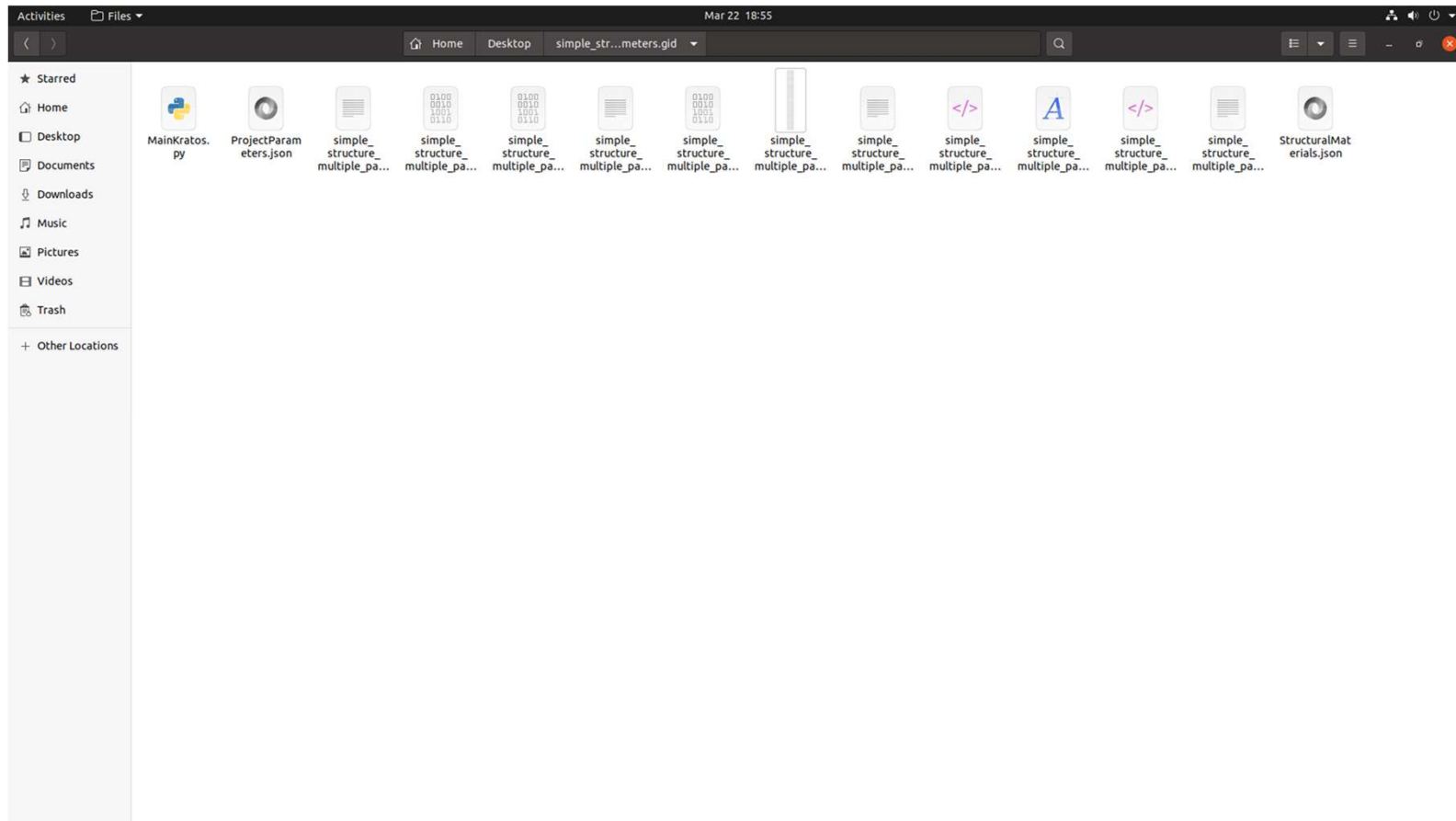
What can be done now (DEMO)?



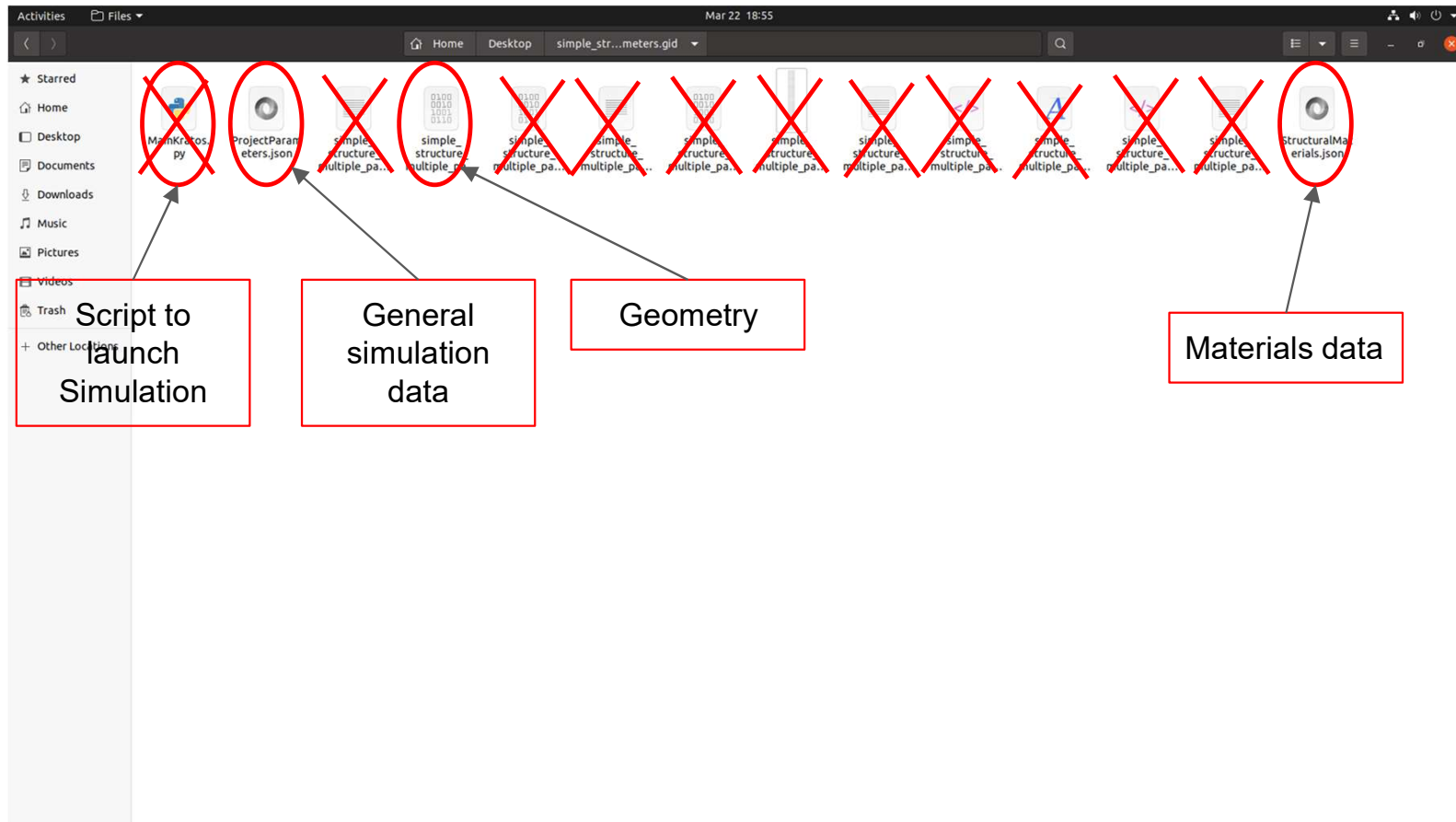
What can be done now (DEMO)?



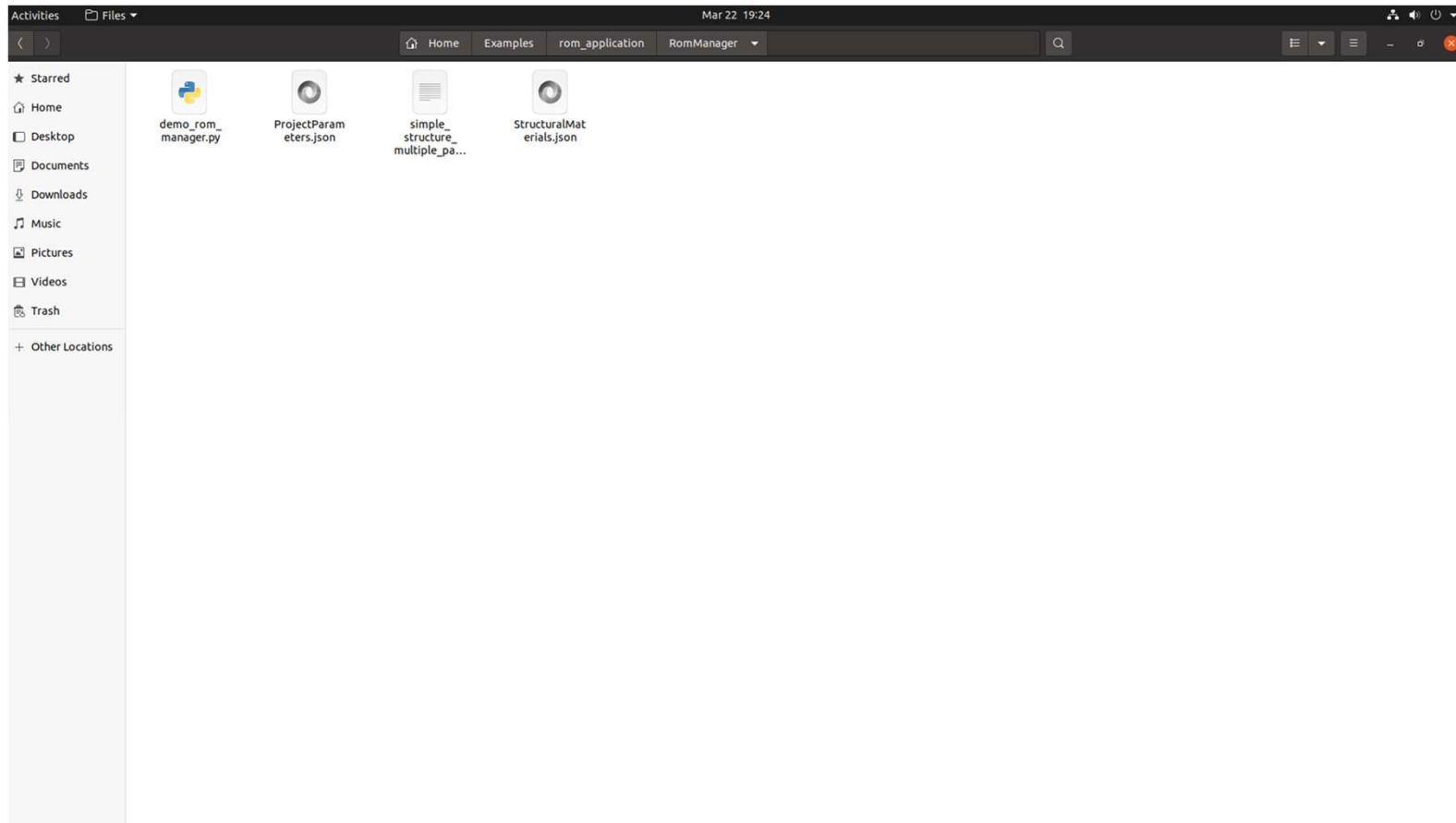
What can be done now (DEMO)?



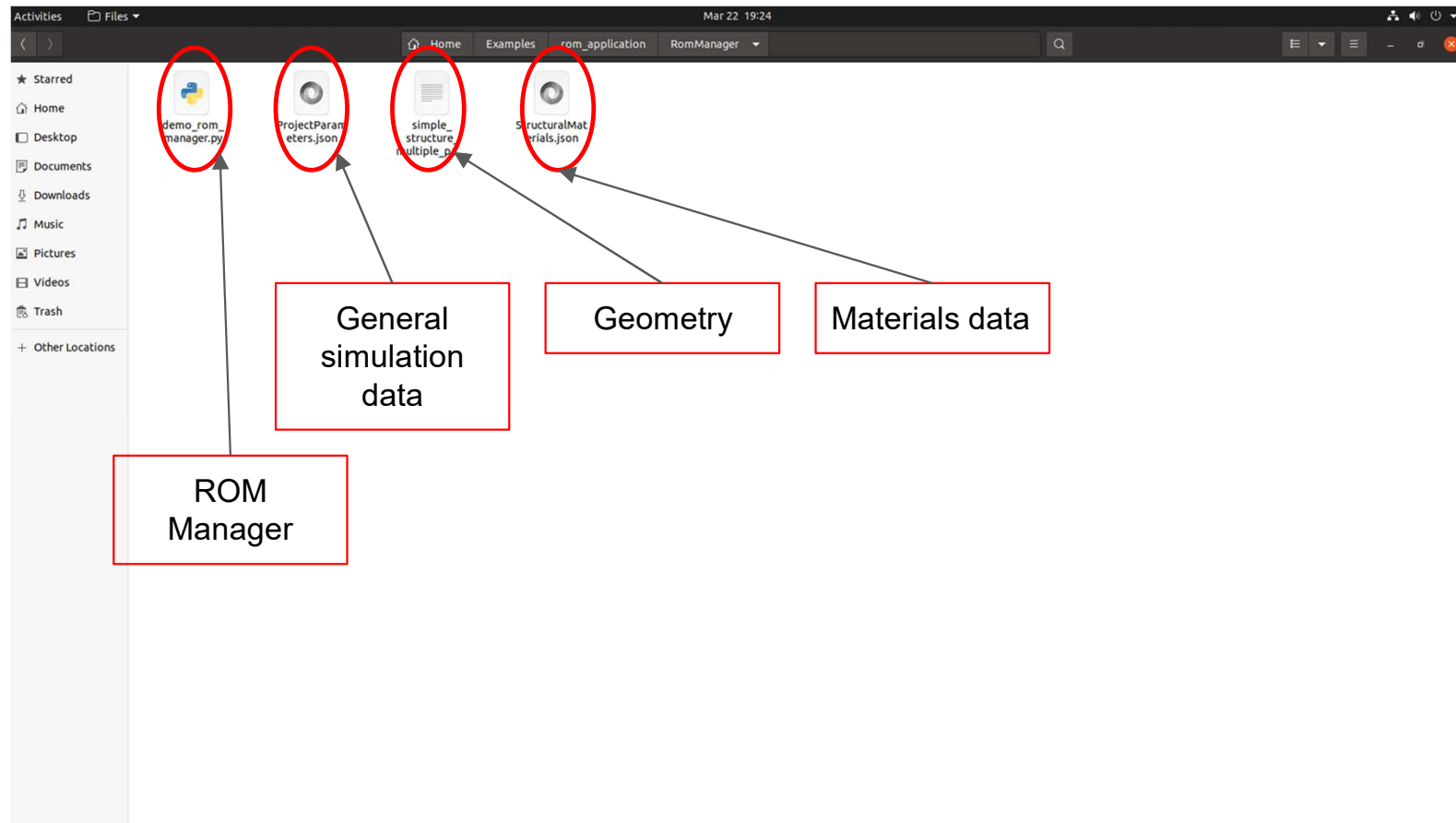
What can be done now (DEMO)?



What can be done now (DEMO)?



What can be done now (DEMO)?



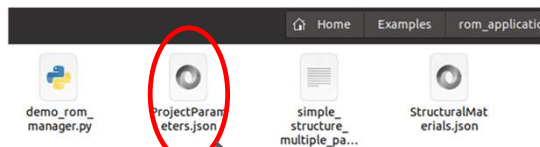
What can be done now (DEMO)?

Customize simulation
class

```
demo_rom_manager.py 2 x {} RomParameters.json
home > jbravo > Examples > rom_application > RomManager > demo_rom_manager.py > GetRomManagerParameters

9
10
11 def CustomizeSimulation(cls, global_model, parameters):
12
13     class CustomSimulation(cls):
14
15         def __init__(self, model, project_parameters, custom_param = None):
16             super().__init__(model, project_parameters)
17             self.custom_param = custom_param
18             """
19             Customize as needed
20             """
21
22         def Initialize(self):
23             super().Initialize()
24             """
25             Customize as needed
26             """
27
28         def FinalizeSolutionStep(self):
29             super().FinalizeSolutionStep()
30             """
31             Customize as needed
32             """
33
34         def CustomMethod(self):
35             """
36             Customize as needed
37             """
38             return self.custom_param
39
40     return CustomSimulation(global_model, parameters)
41
42 #####
43
44 def UpdateProjectParameters(parameters, mu=None):
45     """
46     Customize ProjectParameters here for imposing different conditions to the simulations as needed
47     """
48     parameters["processes"]["loads_process_list"][0]["Parameters"]["value"].SetString("(" + str(mu[0]) + ")")
49     parameters["processes"]["loads_process_list"][1]["Parameters"]["value"].SetString("(" + str(mu[1]) + ")")
50     parameters["processes"]["loads_process_list"][2]["Parameters"]["value"].SetString("(" + str(mu[2]) + ")")
51
52     return parameters
53
54
55
56
```

What can be done now (DEMO)?



General
simulation
data

Customize simulation
parameters

```
demo_rom_manager.py 2 x  {} RomParameters.json
home > jbravo > Examples > rom_application > RomManager > demo_rom_manager.py > GetRomManagerParameters

9
10
11 def CustomizeSimulation(cls, global_model, parameters):
12
13     class CustomSimulation(cls):
14
15         def __init__(self, model, project_parameters, custom_param = None):
16             super().__init__(model, project_parameters)
17             self.custom_param = custom_param
18             """
19             Customize as needed
20             """
21
22         def Initialize(self):
23             super().Initialize()
24             """
25             Customize as needed
26             """
27
28         def FinalizeSolutionStep(self):
29             super().FinalizeSolutionStep()
30             """
31             Customize as needed
32             """
33
34         def CustomMethod(self):
35             """
36             Customize as needed
37             """
38             return self.custom_param
39
40     return CustomSimulation(global_model, parameters)
41
42
43
44
45
46 def UpdateProjectParameters(parameters, mu=None):
47     """
48     Customize ProjectParameters here for imposing different conditions to the simulations as needed
49     """
50     parameters["processes"]["loads_process_list"][0]["Parameters"]["value"].SetString("(" + str(mu[0]) + ")")
51     parameters["processes"]["loads_process_list"][1]["Parameters"]["value"].SetString("(" + str(mu[1]) + ")")
52     parameters["processes"]["loads_process_list"][2]["Parameters"]["value"].SetString("(" + str(mu[2]) + ")")
53
54     return parameters
55
```

What can be done?

DEMO

https://github.com/KratosMultiphysics/Examples/tree/master/rom_application/RomManager

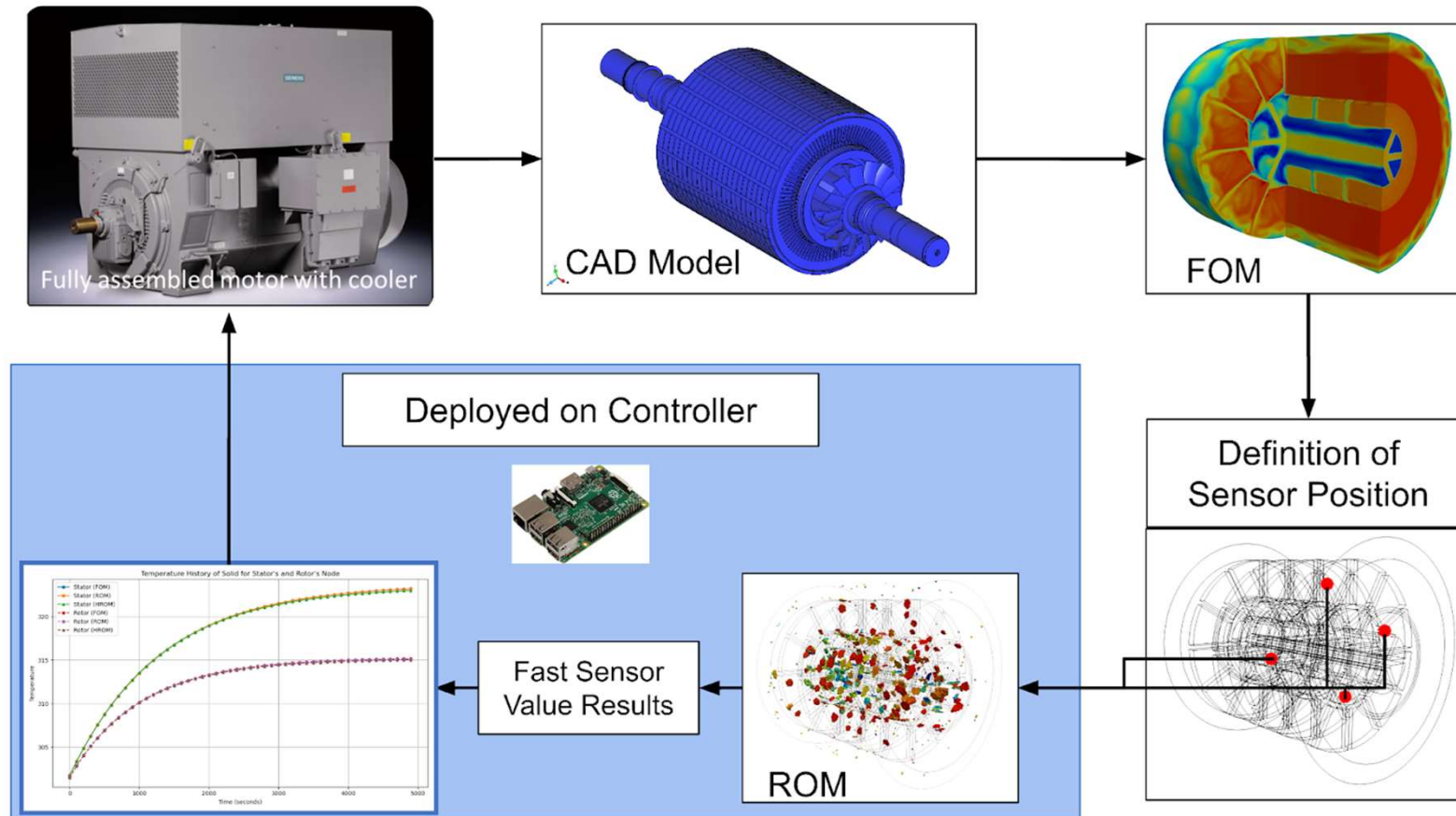
```
pip3 install KratosMultiphysics KratosRomApplication KratosStructuralMechanicsApplication
```

A full tutorial (about 2h of videos):

ROM Manager class to organize the training and the data extraction.



Digital Twins: Manufacturing Industry



ROM Applications



Parallel Reduced Order Modeling for Digital Twins using High-Performance Computing Workflows

S. Ares de Parga^{a,b,*}, J.R. Bravo^{a,b}, N. Sibuet^{a,b}, J.A. Hernandez^{c,b}, R. Rossi^{a,b}, Enrique S. Quintana-Orti^c, Andrés E. Tomás^e, Cristian Cătălin Tatu^d, Fernando Vázquez-Novoa^d, Jorge Ejarque^d, Rosa M Badia^d

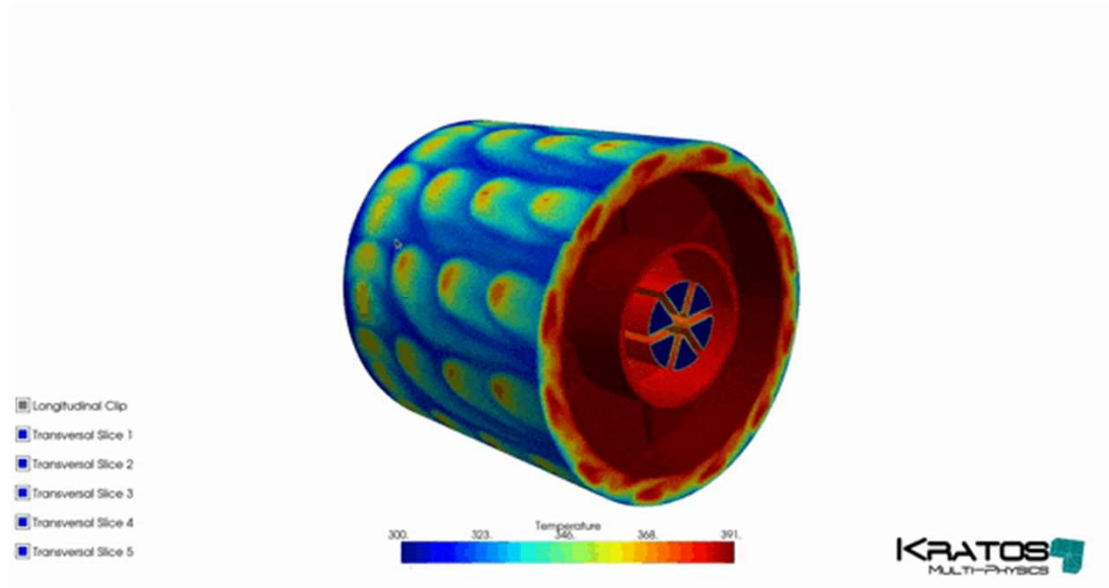
^aDepartment of Civil and Environmental Engineering, Universitat Politècnica de Catalunya, Building B0, Campus Nord, Jordi Girona 1-3, Barcelona 08034, Spain

^bCentre Internacional de Mètodes Numèrics en Enginyeria (CIMNE), Universitat Politècnica de Catalunya, Building C1, Campus Nord, Jordi Girona 1-3, Barcelona 08034, Spain

^cE.S. d'Enginyeries Industrial, Aeroespacial i Audiovisual de Terrassa, Universitat Politècnica de Catalunya, C/ Colom, 11, Terrassa 08222, Spain

^dBarcelona Supercomputing Center, Plaça Eusebi Güell 1-3, Barcelona 08034, Spain

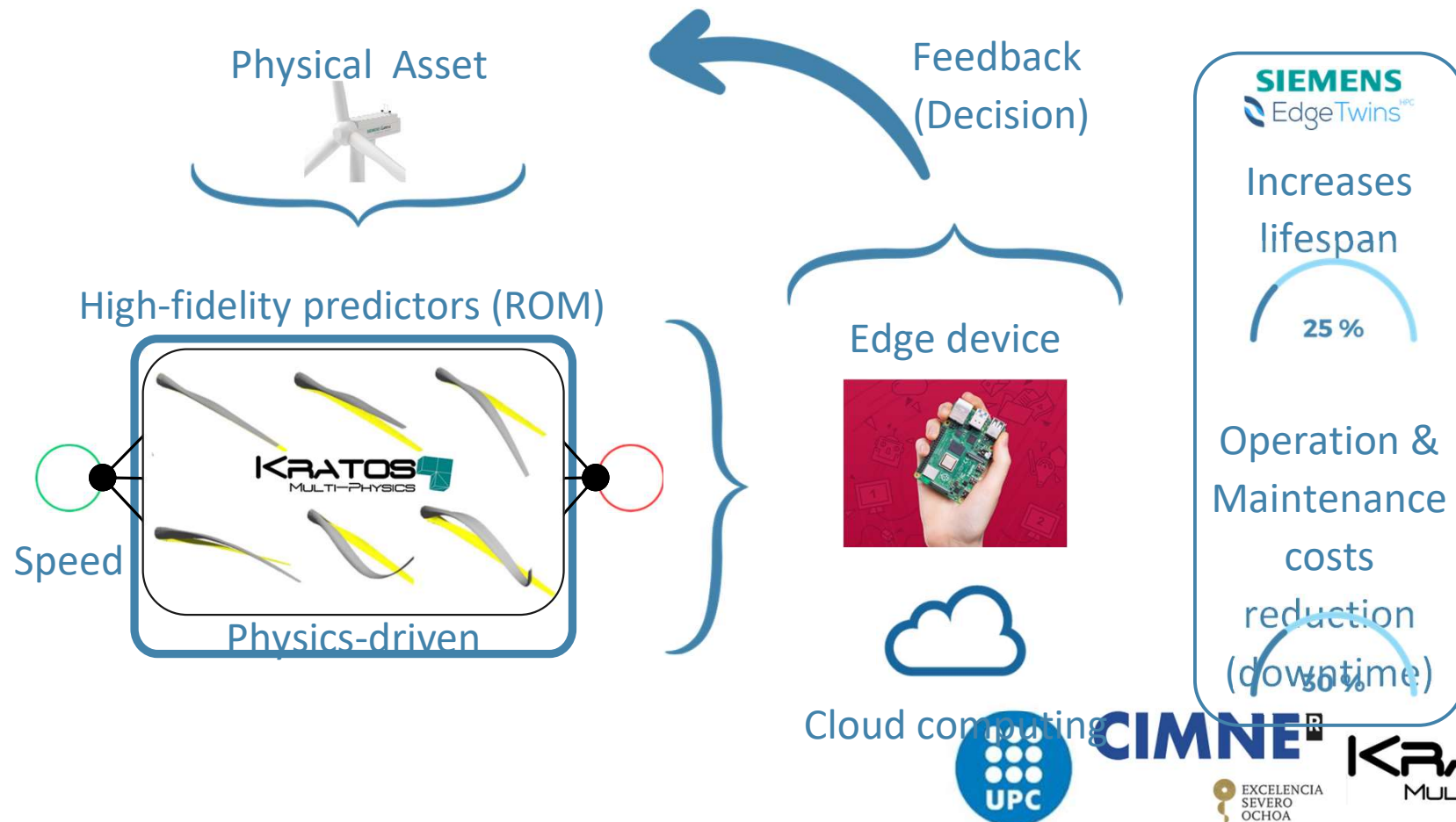
^eDepartamento de Informática de Sistemas y Computadores, Universitat Politècnica de València, Valencia 46022, Spain



ROM Applications: Wind Turbines

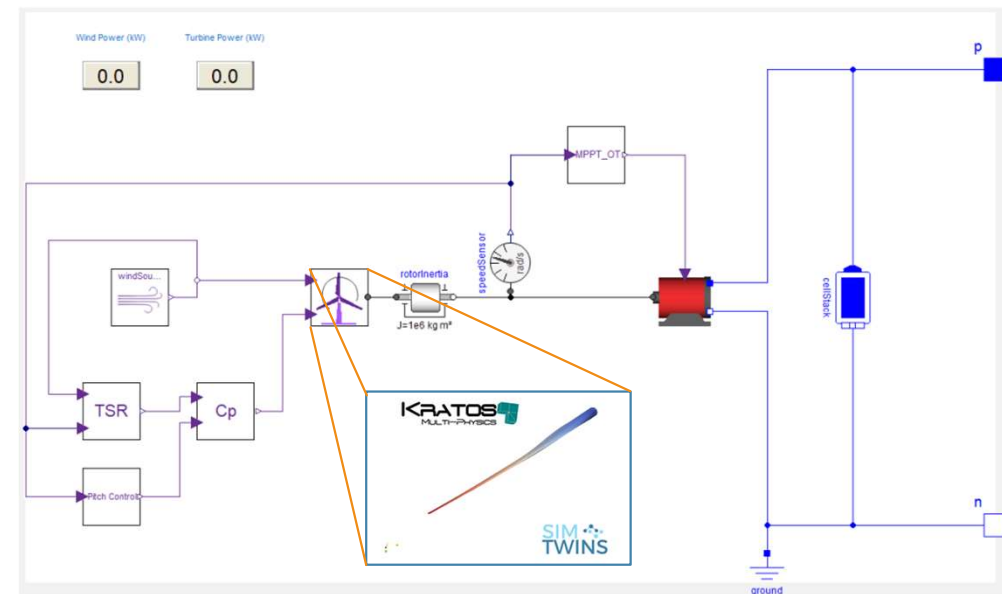


Digital Twins: Wind Engineering Industry



ROM Applications: Wind turbine blade

Digital Twin: Wind Turbine Blade



Some miscellanea ...



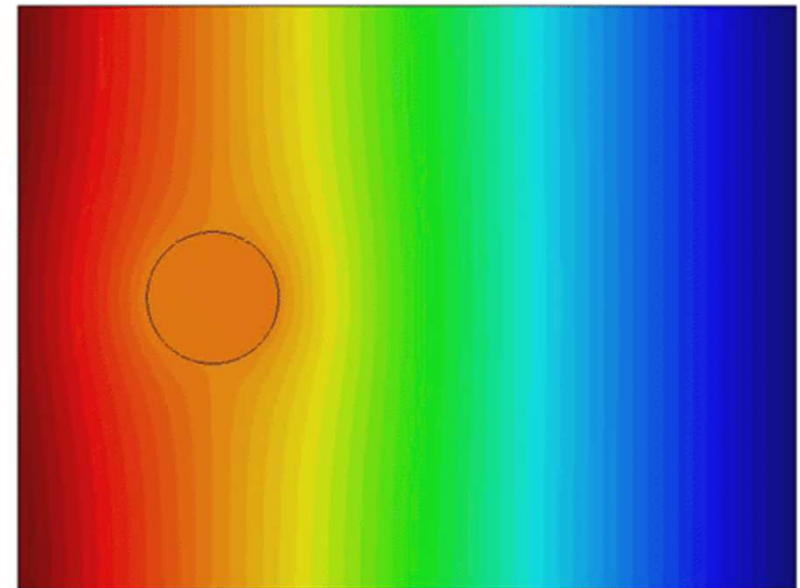
ROM Applications: Droplet Dynamics

Data-driven ROM



Droplet Dynamics - Kratos Multiphysics. AI-aided Computational Solutions for Microfluidics.
Retrieved from <https://droplets.cimne.com/>.

Simulation-driven ROM

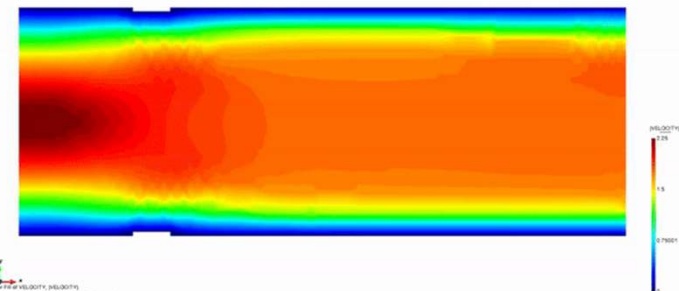
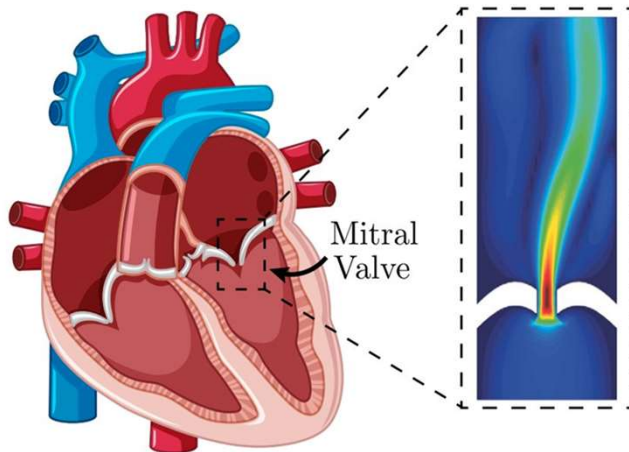
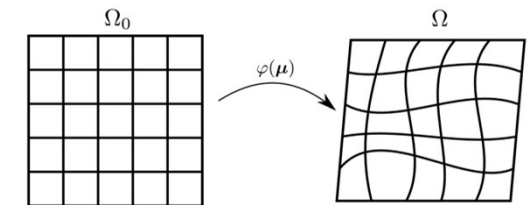
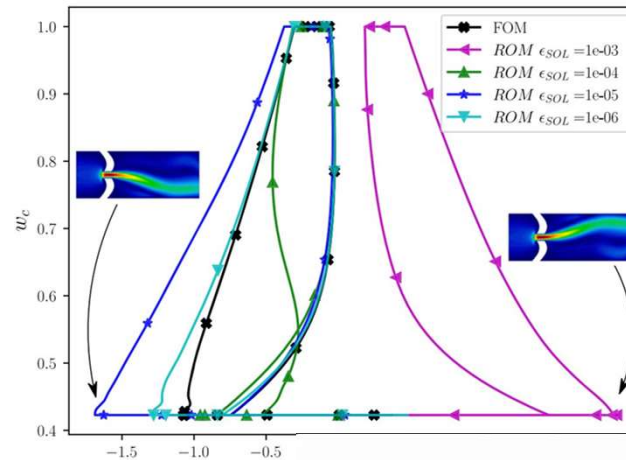


ROM Applications: Geometric Parametrization



Geometrically parametrised reduced order models for studying the hysteresis of the Coanda effect in finite element-based incompressible fluid dynamics

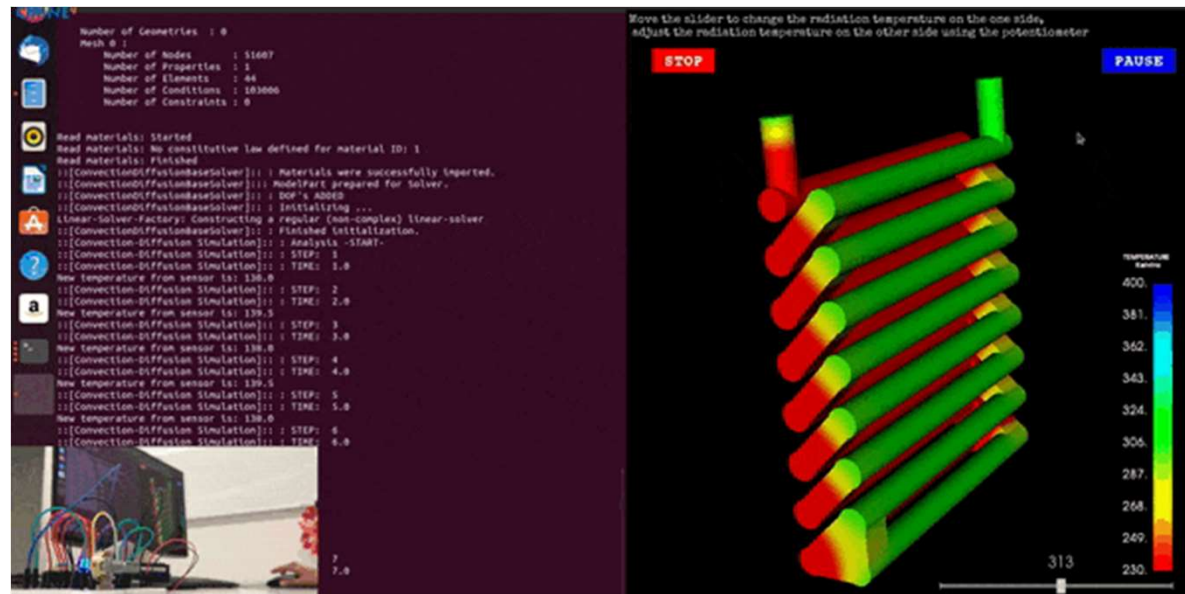
J.R. Bravo ^{a,d,*}, G. Stabile ^c, M. Hess ^b, J.A. Hernandez ^{c,d}, R. Rossi ^{a,d}, G. Rozza ^b



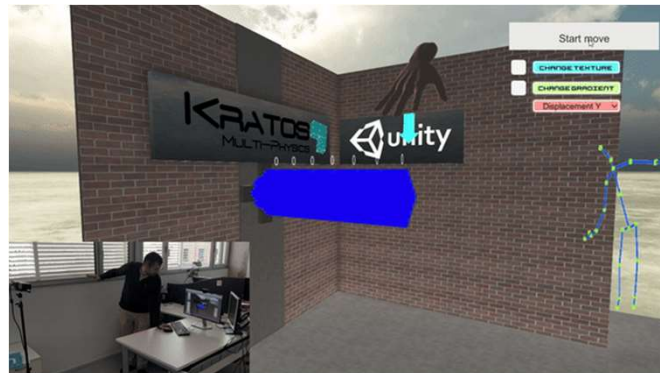
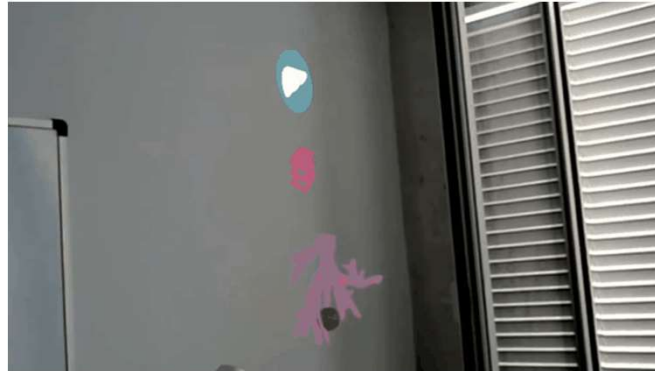
Up to 150x speed-up w.r.t FOM



ROM Applications: Radiator



ROM Applications: Health & AR



THE END

